

# Mastering the Jazzy RMW

ROSCon 2026 | Half Day Workshop

Tuesday, September 22, 1pm - 5pm

*Leave able to prove, with data, which RMW fits your system.*



## | Before we get started ...

- git clone <https://github.com/clearpathrobotics/roscon2026-mastering-the-jazzy-rmw.git>

(or git pull to update if you have the repo already)

- scripts/workshop pull
- scripts/workshop preflight



# Presenters



**Nathan Van Heyst**

Technical Account Executive



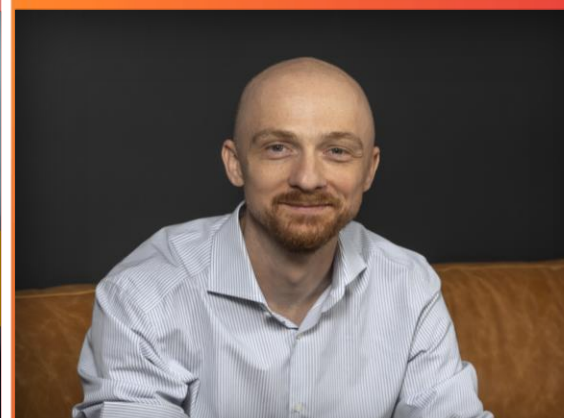
**Luis Camero**

Senior Software Developer



**Matt Bells**

Staff Software Developer, OS



**Andrei Ionescu**

Staff Software Developer,  
Core



# | Workshop Flow

 Intro: RMW foundations & Jazzy landscape

Nathan Van Heyst

 Lab 1: Configuration & tuning

Luis Camero

 Lab 2: ROS 2 On The Wire

Matt Bells

 Lab 3: Stress Testing

Andrei Ionescu

 Lab 4: Benchmarking and Decision Matrix

Andrei Ionescu

 Wrap-up: Summary and Q&A

Nathan Van Heyst

 | You'll leave able to prove your RMW choice.

Choose an RMW with evidence, not defaults.

See what your middleware does on the wire and under stress.

Repeat the method on your own system.



# RMW Foundations & Jazzy Landscape



# | From connectivity to predictable performance

## **2024 Demystifying ROS 2 Networking**

- Discovery
- Network design
- QOS

## **2026 Mastering the RMW**

- Selection
- Configuration
- Benchmarking
- Performance



# | Why Defaults Fail

Defaults are a starting point, not a deployment strategy

## **Workstation**

- Small graph
- Wired or local communication
- Moderate data rates

## **Production Robot**

- Sensor-heavy graph
- Wireless links
- Mixed message sizes and QoS

## **Fleet or Distributed System**

- More participants
- Changing topology
- Aggregate traffic and discovery load



# | ROS 1 vs ROS 2 Myth

"We moved to ROS 2 and latency increased."

This often measures tuned versus untuned, not simply ROS 1 versus ROS 2. The observation could be real, but the framework alone doesn't explain the difference.

## **Tuned ROS 1**

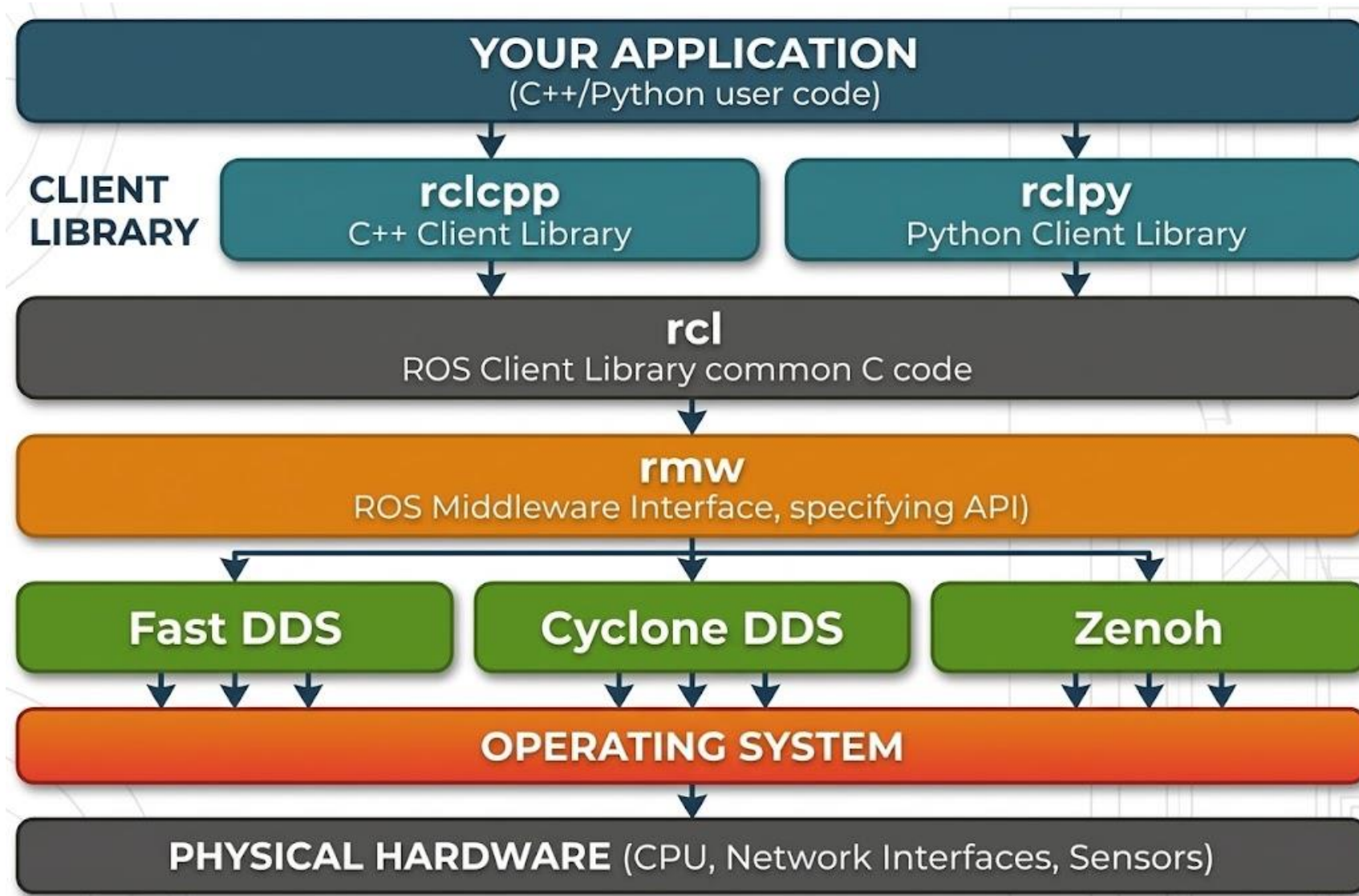
- Nodelets colocated in one process
- Reduced serialization and copying
- Application-specific tuning

## **Untuned ROS 2**

- Nodes launched as separate processes
- Standard middleware communication path
- Default QoS and RMW settings
- Composition and intra-process communication not enabled



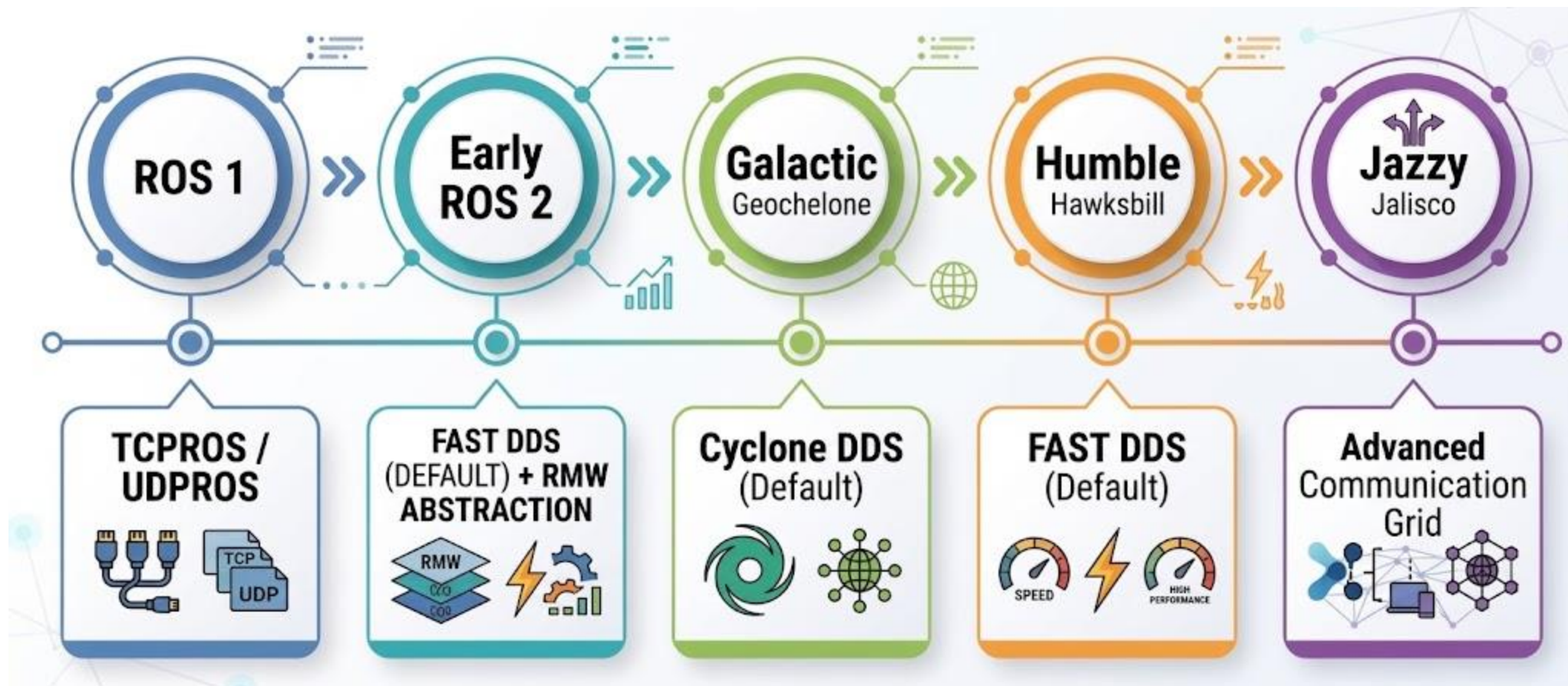
# RMW Anatomy





# RMW History

ROS-specific transport → DDS abstraction → multiple RMW choices → non-DDS options





### The Jazzy default

- DDS/RTPS based
- Included with the ROS 2 binary installation
- Extensive middleware-specific configuration
- Our baseline implementation



### Supported DDS alternative

- DDS/RTPS based
- Available through packaged ROS binaries
- Different implementation and configuration behaviour
- Useful for direct DDS-to-DDS comparison



### Non-DDS RMW option

- Based on the Zenoh protocol
- Router-centred discovery in the documented default setup
- Different topology and communication model
- Useful when evaluating routed or heterogeneous environments



# | How should you choose an RMW?

What does the system need to do reliably?

- **Topology** — single host, robot, fleet, routed sites
- **Network** — wired, wireless, lossy, multicast-constrained
- **Payload** — size, rate, burstiness
- **Performance requirements** — latency, delivery, jitter
- **Scale** — nodes, topics, participants, robots
- **Resources** — CPU, memory, bandwidth

# | How does this connect to the labs?

Configure → Measure → Stress → Decide

**Configure** — Select and tune the RMW

**Measure** — Capture performance and resource use

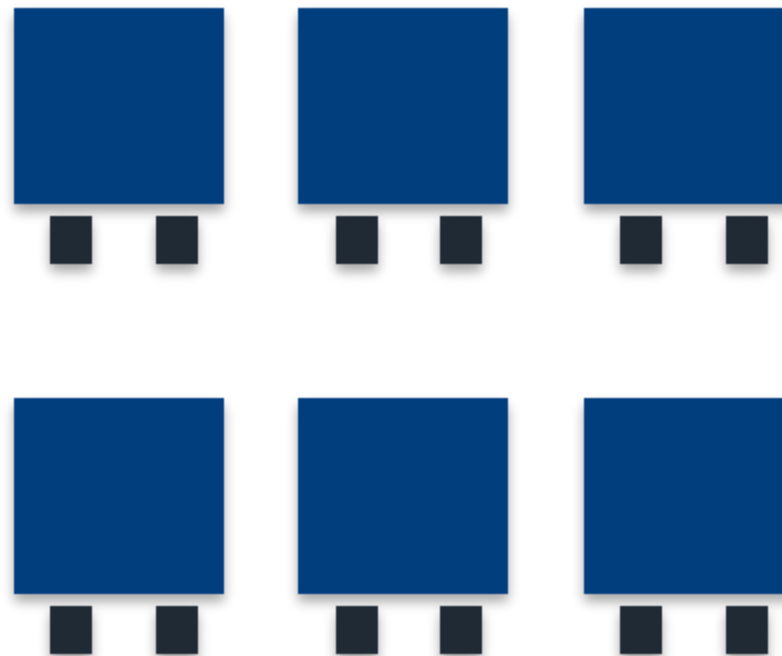
**Stress** — Apply realistic loads and scale

**Decide** — Compare evidence and trade-offs



# | A synthetic fleet gives everyone the same workload.

- a full fleet per laptop
- identical scenario for everyone
- repeatable traffic



 | netem makes the network impairment repeatable.

**Synthetic fleet**



**netem: delay, loss, rate**



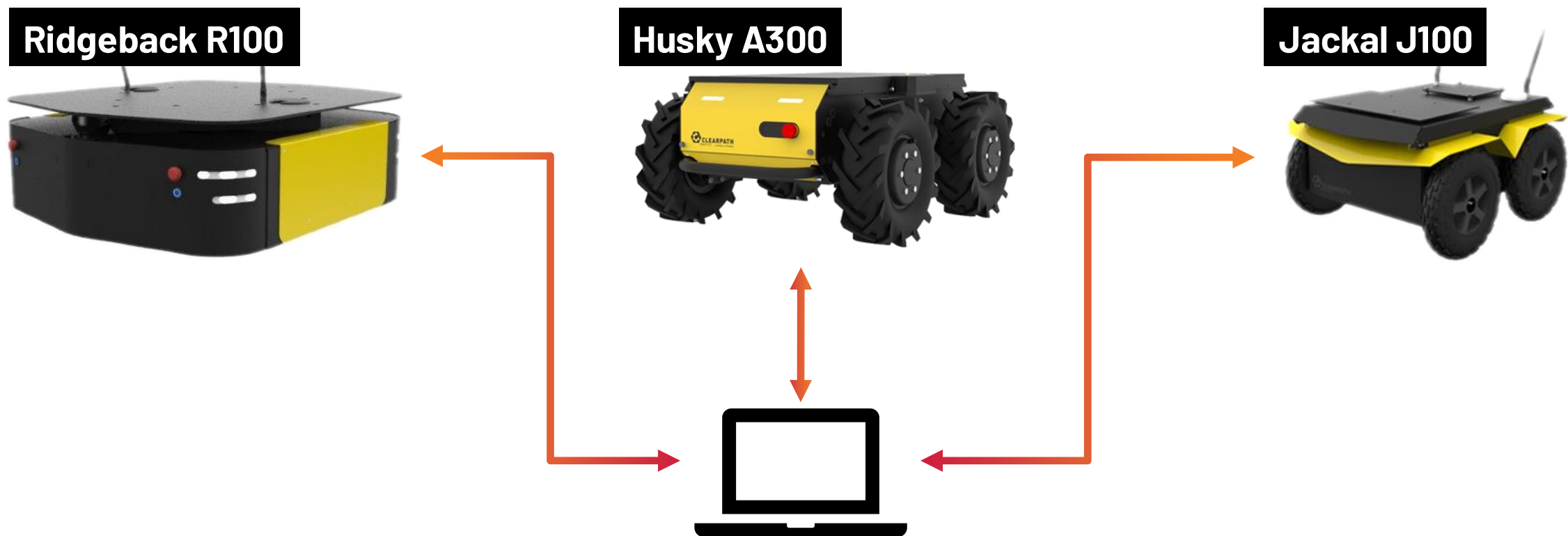
**Observe RMW behaviour**



# Lab 1: Configuration & Tuning

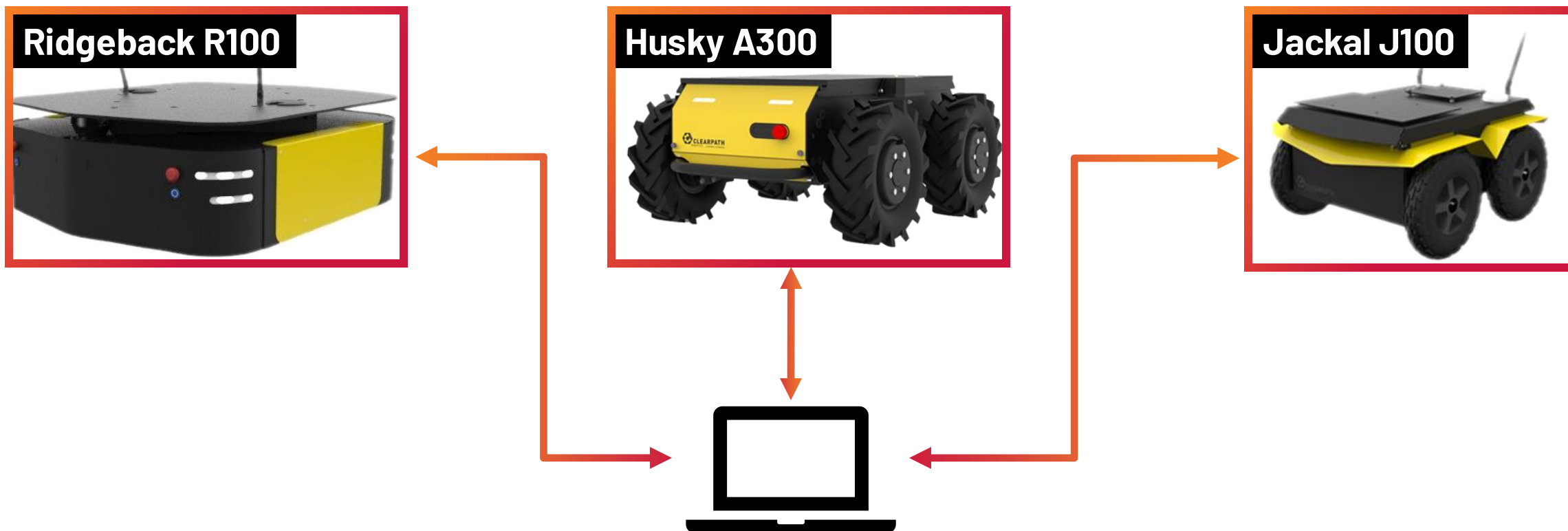
# | Introduction to the Workshop Containers

How do we bring a fleet of robots to a ROSCon workshop?



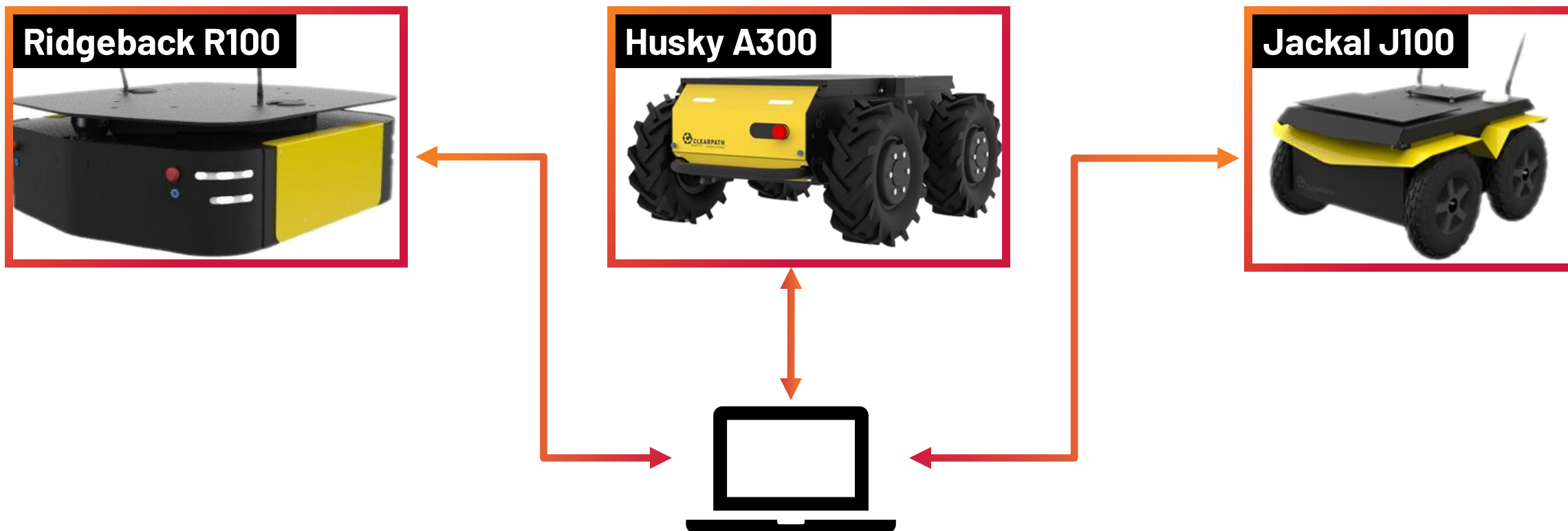
# | Introduction to the Workshop Containers

Put them in Docker containers!



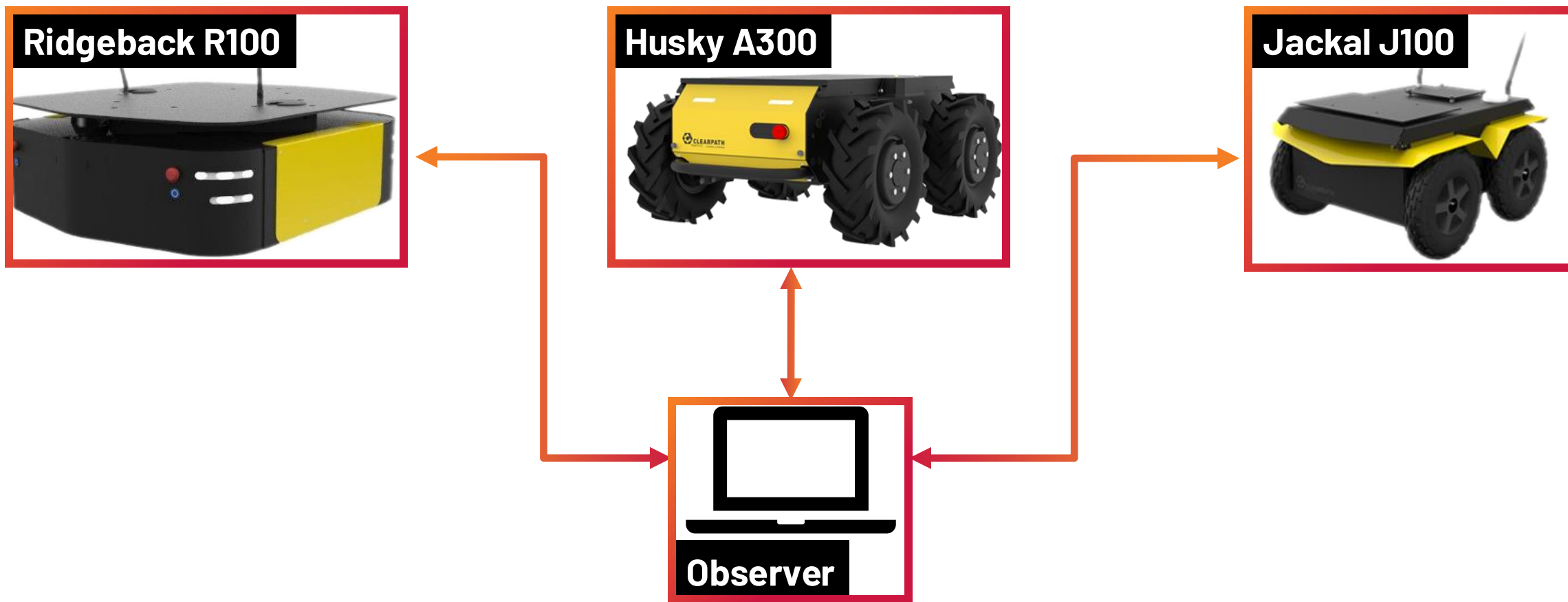
# | Introduction to the Workshop Containers

But, what about networking between the user computer and the robots?

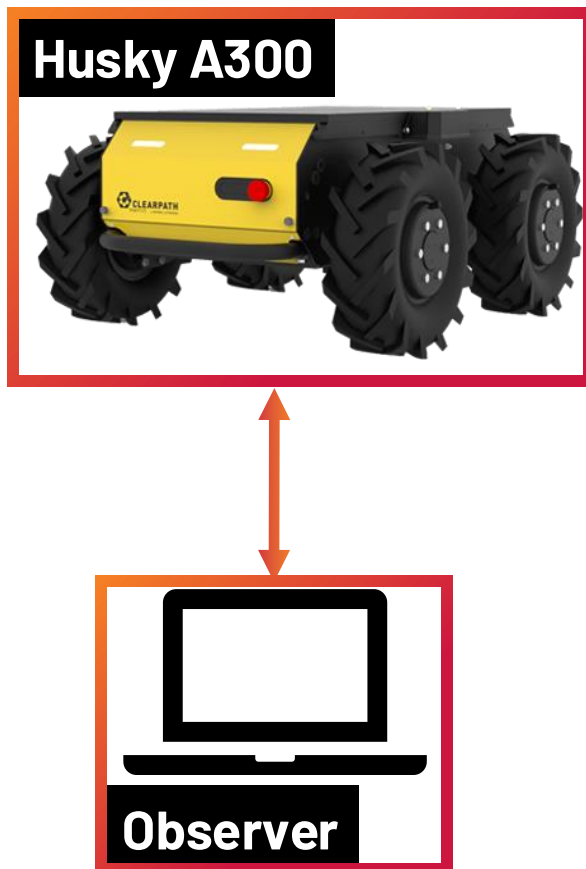


# | Introduction to the Workshop Containers

Put it in a docker, too!



# RA | Exercise 1: Bring up your first robot



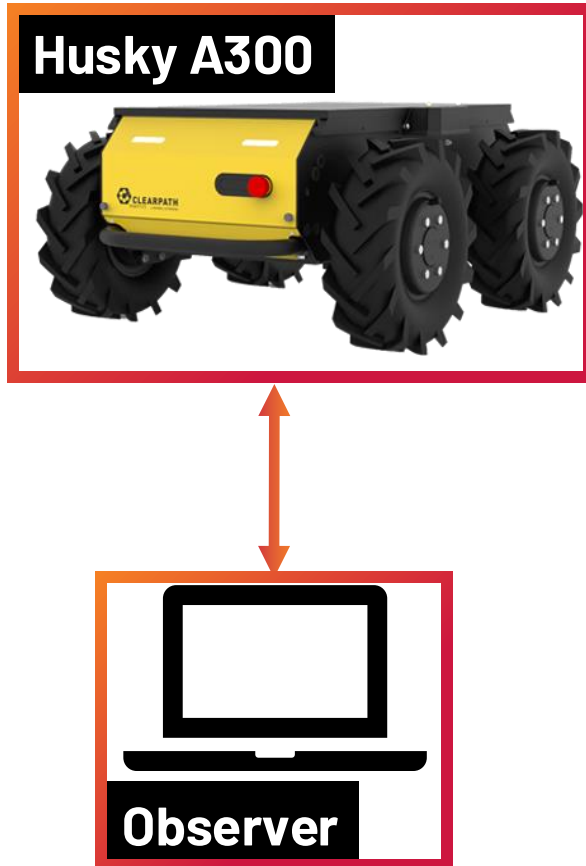
To familiarize ourselves with the building blocks of this workshop, we will bring up:

- one robot,
- one observer, and
- a visualization UI.

In the workshop repository, open:

**Exercise 1: Bring up your first robot**  
(*1\_bring\_up\_your\_first\_robot.md*)

# RA | Exercise 1: Bring up your first robot

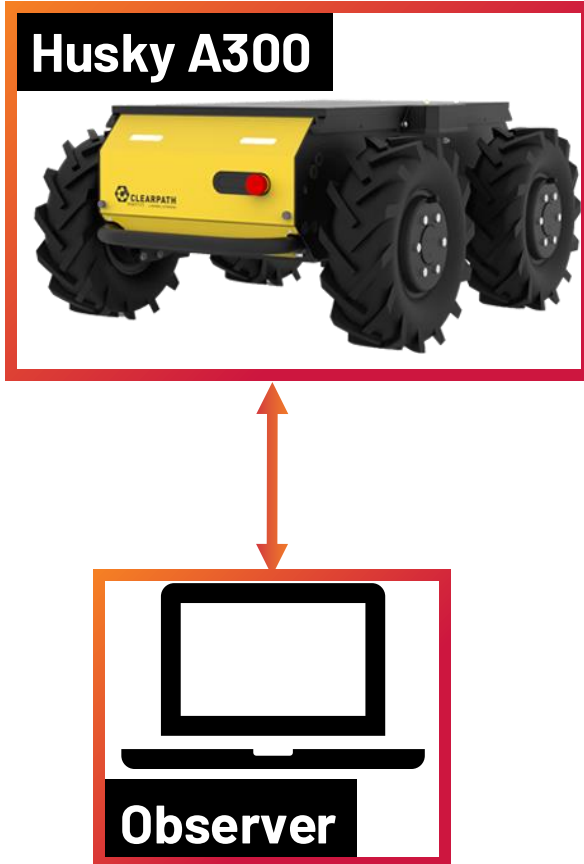


**Step 1:** Use the workshop script to bring up a single robot, single observer:

```
MOCK_RUN_PILOT=false scripts/workshop -t flat up 1
```

- `scripts/workshop`: common entrypoint
- `-t flat`: topology flat, one shared network, we'll come back to this later.
- `up 1`: bring up one mock robot
- `MOCK_RUN_PILOT=false`: Disable the automated driving of the robot. This will come useful in later labs.

# RA | Exercise 1: Bring up your first robot



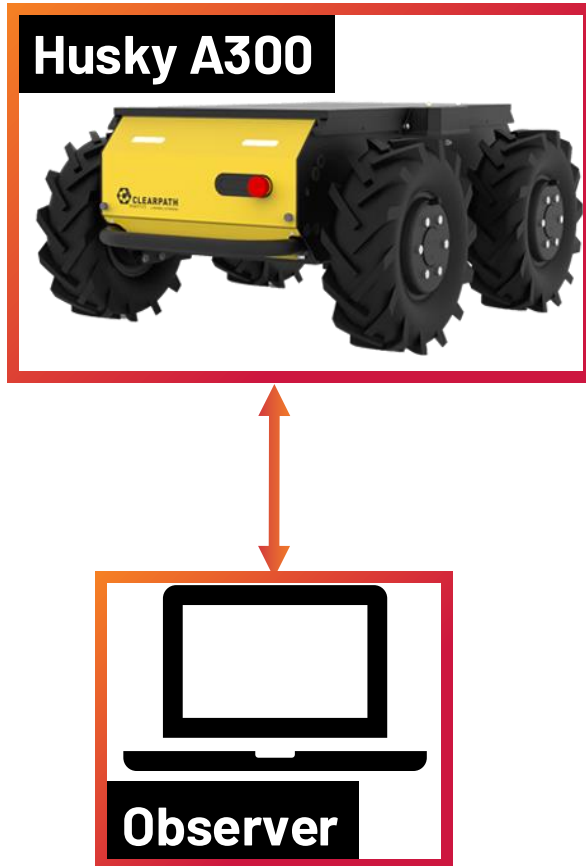
**Step 2:** Start the foxglove bridge on the **observer** container

```
scripts/workshop observer bridge
```

| Property      | Details                                                           |
|---------------|-------------------------------------------------------------------|
| Image         | Ubuntu 24.04 Headless                                             |
| Namespace     | <b>observer</b>                                                   |
| Default state | Idle, waits for users to run tools (e.g. foxglove <b>bridge</b> ) |

The **observer** is a stand-in for a user/fleet manager/monitor on the network. Think of it as your laptop when trying to communicate with an existing robot(s).

# RA | Exercise 1: Bring up your first robot



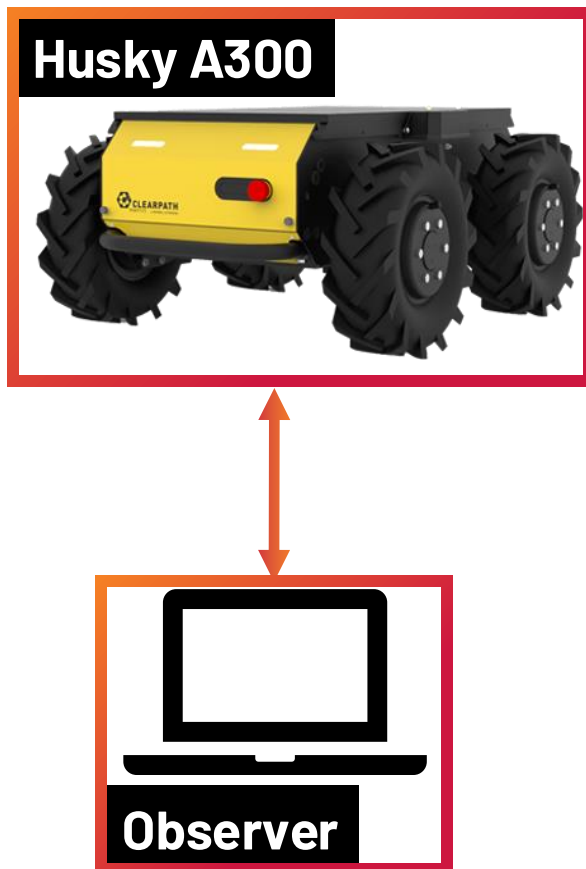
**Step 3:** Start the visualization UI, the **lichtblick** container

```
scripts/workshop lichtblick up
```

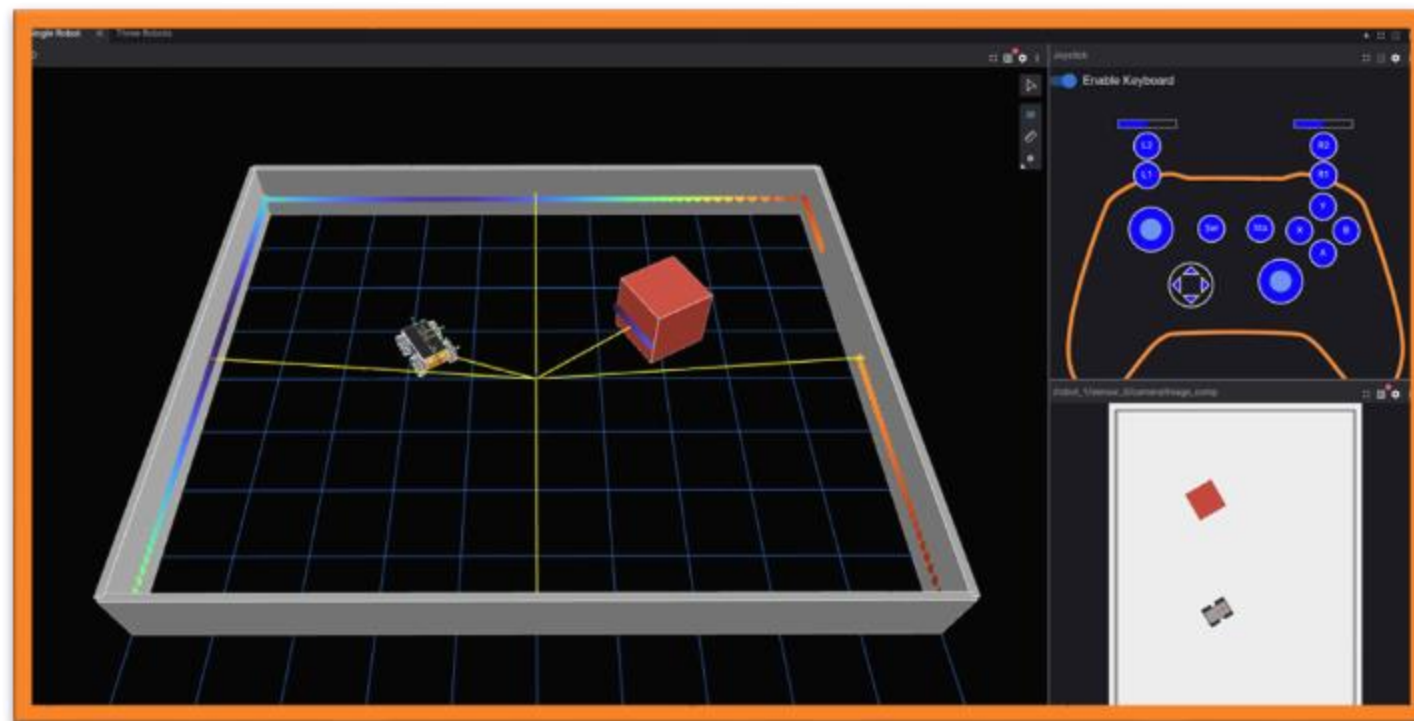
| Property     | Details                                     |
|--------------|---------------------------------------------|
| Image        | lichtblick                                  |
| Extensions   | <b>Joshnewans.joy-panel</b> (gamepad panel) |
| Connects via | The foxglove bridge on the container        |

The **lichtblick** web UI is a stand-in for any visualization tool you prefer. In your own work, you might prefer to use Foxglove, RViz, or other custom visualization tools.

# RA | Exercise 1: Bring up your first robot

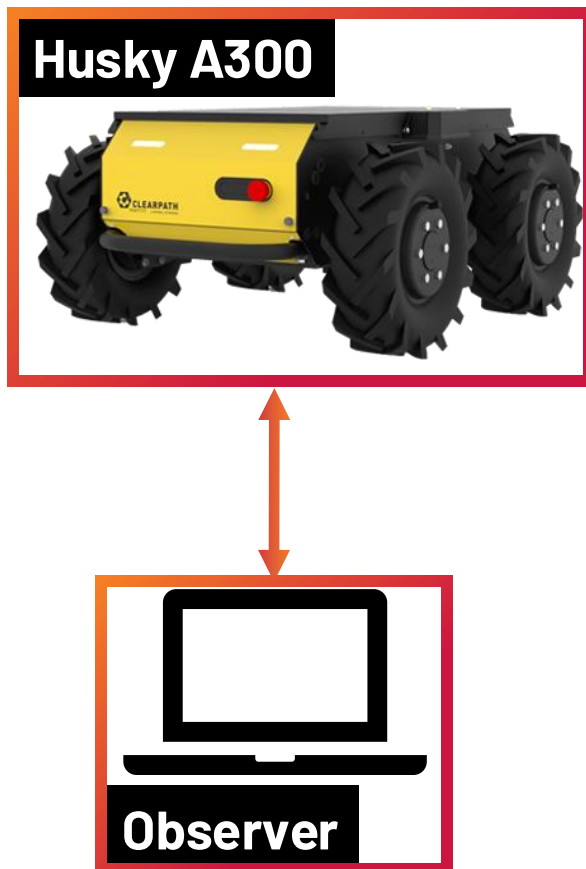


**Step 4:** Open a browser to visualize the robot you brought up.





## | Exercise 2: Inspect the robot and its QoS



To familiarize ourselves with the nodes and topics on the mock robot, the ROS CLI, and a quick refresher on the Quality of Service of each topic.

In the workshop repository, open:

**Exercise 2: Inspect the robot and its QoS**  
(*2\_inspect\_the\_robot\_and\_its\_qos.md*)



## | Exercise 2: Inspect the robot and its QoS

**Husky A300**



**Observer**

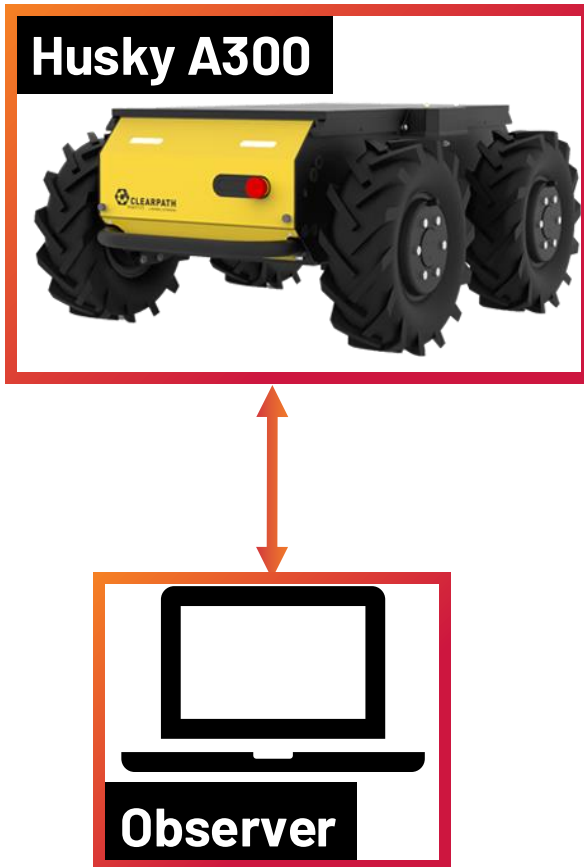
**Step 1:** Use the workshop script to bring up a single robot, single observer (if you brought it down), then open a shell on the robot container:

```
MOCK_RUN_PILOT=false scripts/workshop -t flat up 1  
Scripts/workshop shell mock-robot-1
```

**Steps 2&3:** From within the robot container, list the robot's nodes and topics.

```
ros2 node list  
ros2 topic list
```

# RA | Exercise 1: Bring up your first robot



**Step 1:** Use the workshop script to bring up a single robot, single observer:

```
MOCK_RUN_PILOT=false scripts/workshop -t flat up 1
```

| Property    | Details                                      |
|-------------|----------------------------------------------|
| Image       | Ubuntu 24.04 Headless                        |
| Namespace   | <b>robot_&lt;n&gt;</b> (robot_1)             |
| Robot Model | Husky a300                                   |
| Topics      | <ns>/camera/image_raw, <ns>/scan, /tf, etc.  |
| Middleware  | Selectable ( <i>rmw_cyclone_dds</i> default) |



## | Exercise 2: Inspect the robot and its QoS

But what is a QoS?

| Policy         | Options                                       | Details                                                                                     |
|----------------|-----------------------------------------------|---------------------------------------------------------------------------------------------|
| History        | <b>KEEP_LAST</b> (depth N) or <b>KEEP_ALL</b> | How many samples are queued before delivery.                                                |
| Depth          | int                                           | Queue size, used only with <b>KEEP_LAST</b> .                                               |
| Reliability    | <b>RELIABLE</b> or <b>BEST_EFFORT</b>         | <b>RELIABLE</b> retransmits until confirmed, <b>BEST_EFFORT</b> allows dropped samples.     |
| Durability     | <b>VOLATILE</b> or <b>TRANSIENT_LOCAL</b>     | <b>TRANSIENT_LOCAL</b> re-delivers last samples to late joiners<br><b>VOLATILE</b> does not |
| Deadline       | duration                                      | Maximum expected gap between messages.                                                      |
| Lifespan       | duration                                      | How long a sample stays valid before its dropped                                            |
| Liveliness     | <b>AUTOMATIC</b> or <b>MANUAL_BY_TOPIC</b>    | How a publisher is judged "still alive"                                                     |
| Lease Duration | duration                                      | Time within which a publisher must assert liveliness                                        |



## | Exercise 2: Inspect the robot and its QoS

**Husky A300**



**Observer**

**Step 4a:** Use the topic info to print the offered/requested QoS of the **robot description topic**.

```
ros2 topic info -v /robot_1/robot_description
```

You should see the following:

QoS profile:

Reliability: RELIABLE

History (Depth): KEEP\_LAST (1)

Durability: **TRANSIENT\_LOCAL**

Lifespan: Infinite

Deadline: Infinite

Liveliness: AUTOMATIC

Liveliness lease duration: Infinite



## | Exercise 2: Inspect the robot and its QoS

**Husky A300**



**Observer**

**Step 4b:** Use the topic info to print the offered/requested QoS of the **joint states** .

```
ros2 topic info -v /robot_1/joint_states
```

You should see the following:

QoS profile:

Reliability: RELIABLE

History (Depth): KEEP\_LAST (1)

Durability: **VOLATILE**

Lifespan: Infinite

Deadline: Infinite

Liveliness: AUTOMATIC

Liveliness lease duration: Infinite



## | Exercise 2: Inspect the robot and its QoS

**Husky A300**



**Observer**

**Step 4c:** Use the topic info to print the offered/requested QoS of the **image** topic.

```
ros2 topic info -v  
/robot_1/sensor_0/camera/image_raw
```

You should see the following:

QoS profile:

Reliability: **RELIABLE**

History (Depth): KEEP\_LAST (10)

Durability: VOLATILE

Lifespan: Infinite

Deadline: Infinite

Liveliness: AUTOMATIC

Liveliness lease duration: Infinite

Does it make sense for it to be set to reliable?



## | Exercise 2: Inspect the robot and its QoS

**Husky A300**



**Observer**

**Step 4c:** Use the topic bandwidth to print the bandwidth of the **image** topic.

```
ros2 topic bw /robot_1/sensor_0/camera/image_raw
```

You should see the following:

```
Subscribed to [/robot_1/sensor_0/camera/image_raw]
```

```
27.65 MB/s from 10 messages
```

```
Message size mean: 2.68 MB min: 2.68 MB max: 2.68 MB
```

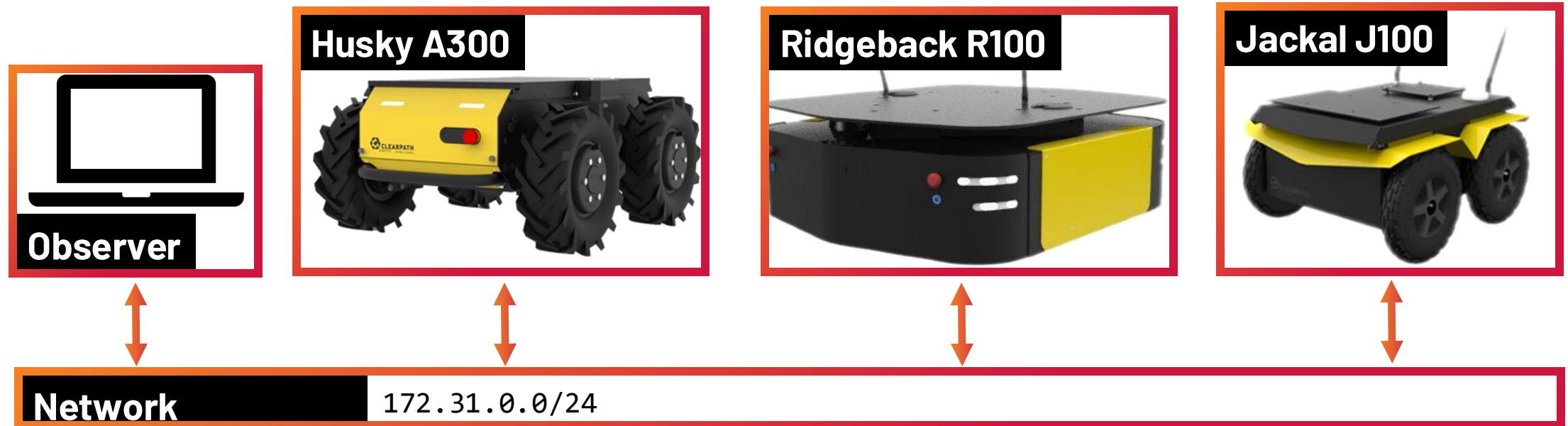
Does it make sense for it to be set to reliable?



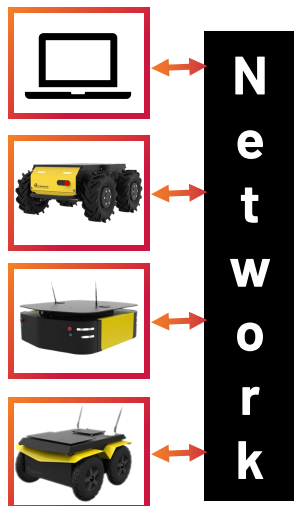
## | Exercise 3: Fleet Discovery and Domain Isolation

In this exercise we will be bringing up 3 robots, where they are all connected to the observer over a shared network bridge.

Open the **Exercise 3: Fleet Discovery and Domain Isolation**  
(*3\_fleet\_discovery\_and\_domains.md*)



# RA | Exercise 3: Fleet Discovery and Domain Isolation



**Step 1:** Bring up **three** robots on a **flat** topology using **Cyclone DDS**.

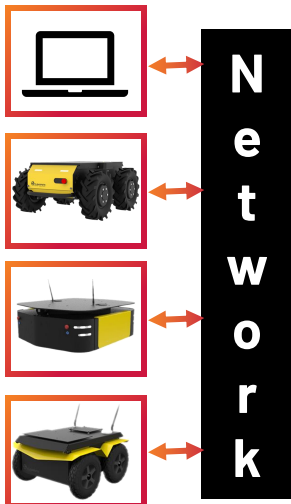
```
MOCK_RUN_PILOT=false scripts/workshop -t flat up 3 cyclone --model  
a300,j100,r100
```

**Step 2:** Confirm every robot sees every other robot's topics:

```
scripts/workshop shell mock-robot-1  
ros2 topic list  
ros2 node list  
exit
```

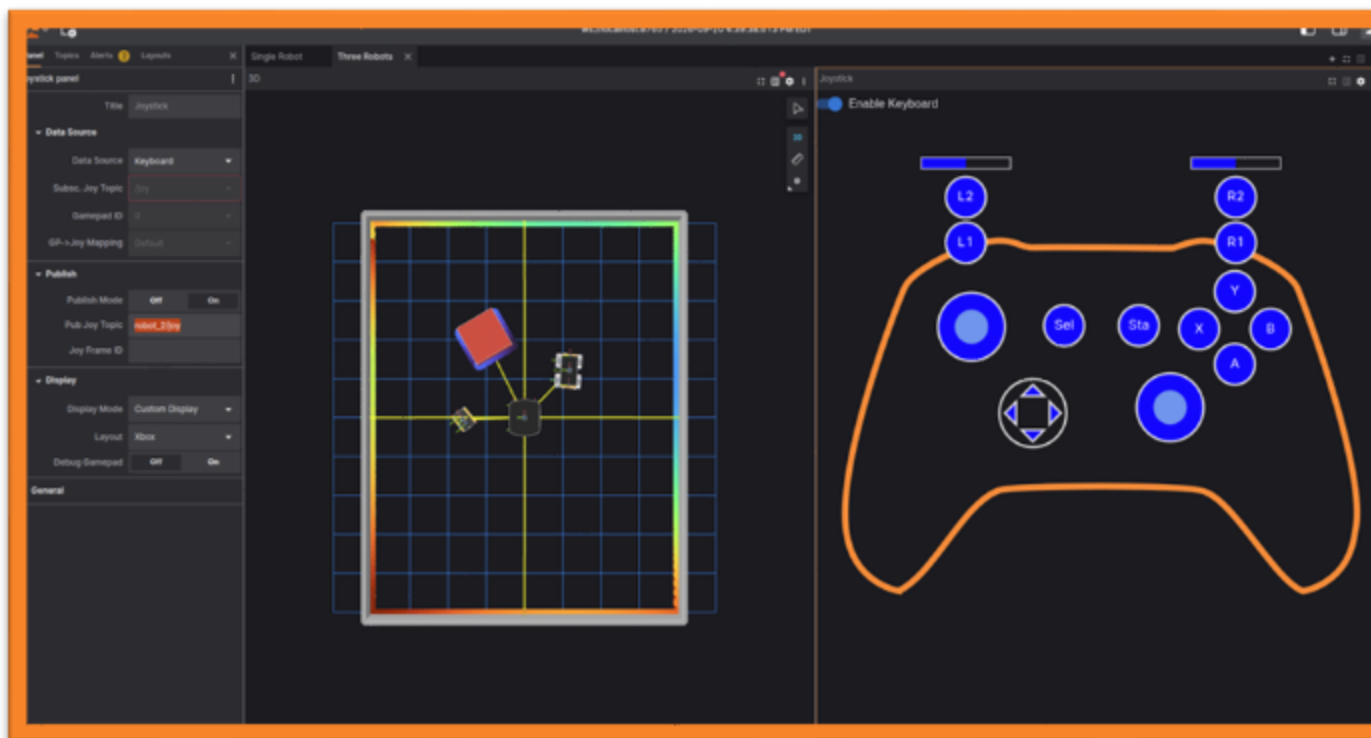


# | Exercise 3: Fleet Discovery and Domain Isolation



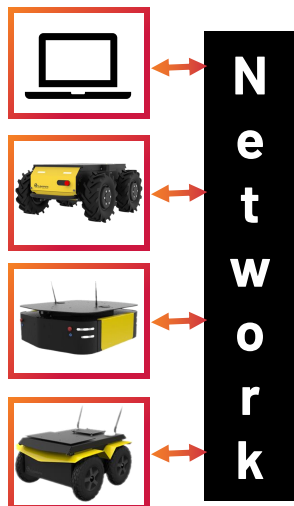
**Step 3:** Optionally, open up lichtblick to see the three robot fleet.

```
scripts/workshop observer bridge  
scripts/workshop lichtblick up
```



Change the **Publisch -> Pub Joy Topic** to select a different robot to drive

# RA | Exercise 3: Fleet Discovery and Domain Isolation



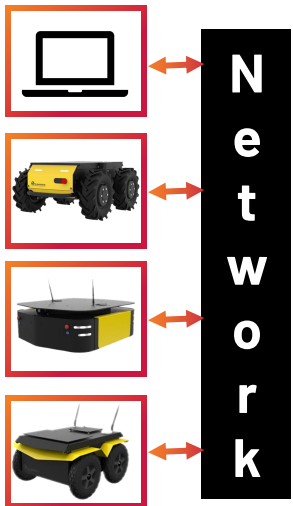
## Step 4: Inspect the RMW configuration for Cyclone DDS

```
cat docker/rmw_configuration/flat/cyclone/mock-robot.xml  
cat docker/rmw_configuration/flat/cyclone/observer.xml
```

You should expect to see:

```
<CycloneDDS>  
  <Domain id="any">  
    <General>  
      <AllowMulticast>true</AllowMulticast>  
    </General>  
    <Discovery>  
      <ParticipantIndex>auto</ParticipantIndex>  
      <MaxAutoParticipantIndex>120</MaxAutoParticipantIndex>  
    </Discovery>  
  </Domain>  
</CycloneDDS>
```

## RA | Exercise 3: Fleet Discovery and Domain Isolation



**Step 5:** Let's split the robots into their own domains, such that they don't discovery each other's topics.

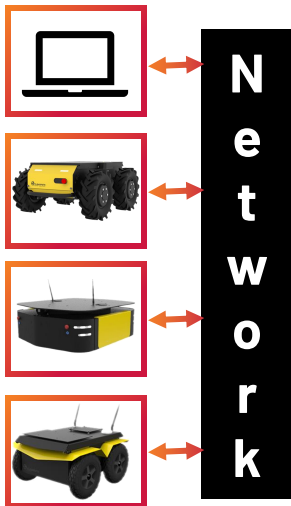
```
scripts/workshop -t flat down
```

An environment variable sets the **ROS\_DOMAIN\_ID**, participants on a specific domain only discover other participants on that same domain. Edit the *docker/compose/flat.yml* and give each robot its own domain:

```
# under mock-robot-1:
  ROS_DOMAIN_ID: "1"
# under mock-robot-2:
  ROS_DOMAIN_ID: "2"
# under mock-robot-3:
  ROS_DOMAIN_ID: "3"
```



# | Exercise 3: Fleet Discovery and Domain Isolation



**Step 5 (cont'd):** Once you've edited the file, bring the fleet back up but make sure to use the **--skip-gen** flag.

```
MOCK_RUN_PILOT=false scripts/workshop -t flat up 3 cyclone --skip-gen
```

**Step 6:** Shell into the observer and hop between domains.

```
scripts/workshop shell observer

export ROS_DOMAIN_ID=1
ros2 node list --no-daemon --spin-time 5      # only robot_1

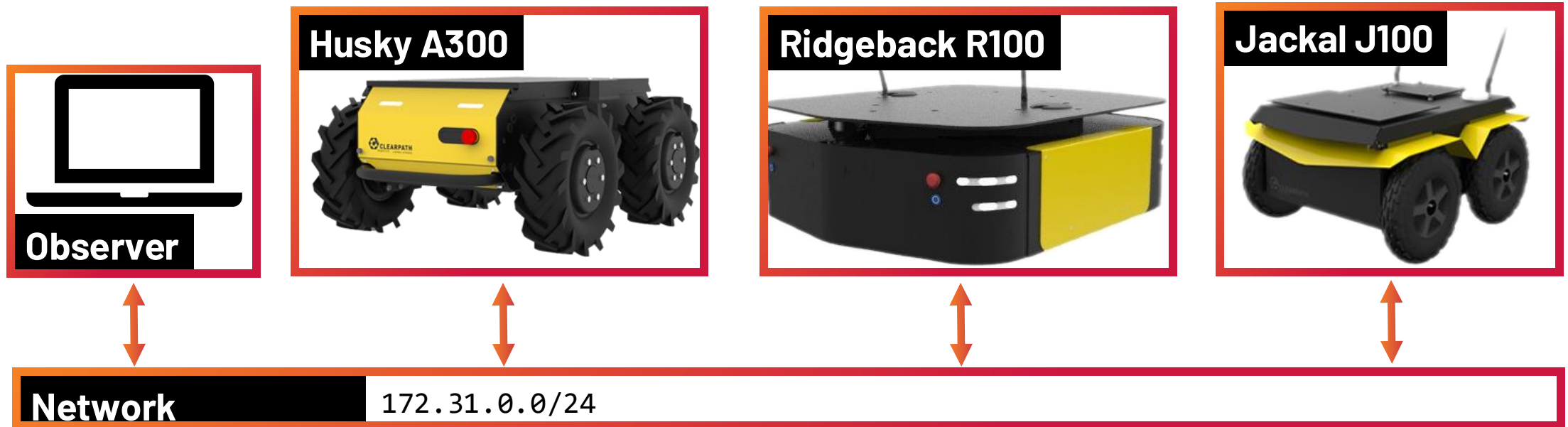
export ROS_DOMAIN_ID=2
ros2 node list --no-daemon --spin-time 5      # only robot_2

export ROS_DOMAIN_ID=3
ros2 node list --no-daemon --spin-time 5      # only robot_3
exit
```



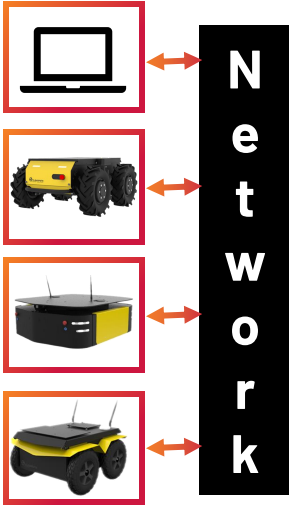
## | Exercise 4: Disable Multicast and Pin Peers

Domains provide a means to isolate discovery, but it does not prevent multicast discovery traffic. Instead, we can disable multicast and fix peers. Open the **Exercise 4: Disable Multicast and Pin Peers** (*4\_disable\_multicast\_and\_pin\_peers.md*)





# | Exercise 4: Disable Multicast and Pin Peers



**Step 1:** Create a directory where we will be creating Cyclone configurations

```
mkdir -p docker/rmw_configuration/lab1/cyclone
```

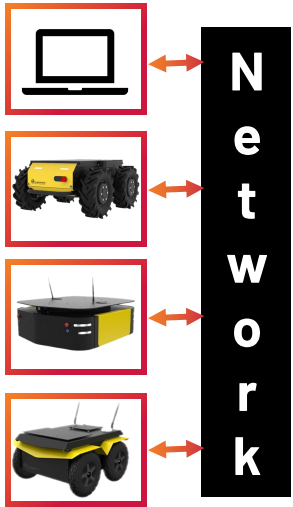
**Step 2:** Create the observer's profile:

```
<CycloneDDS>
  <Domain id="any">
    <General>
      <AllowMulticast>false</AllowMulticast>
    </General>
    <Discovery>
      <ParticipantIndex>auto</ParticipantIndex>
      <MaxAutoParticipantIndex>120</MaxAutoParticipantIndex>
      <Peers>
        <Peer address="localhost"/>
        <Peer address="mock-robot-1"/>
        <Peer address="mock-robot-2"/>
        <Peer address="mock-robot-3"/>
      </Peers>
    </Discovery>
  </Domain>
</CycloneDDS>
```

*docker/rmw\_configuration/lab1/cyclone/observer.xml*



# | Exercise 4: Disable Multicast and Pin Peers



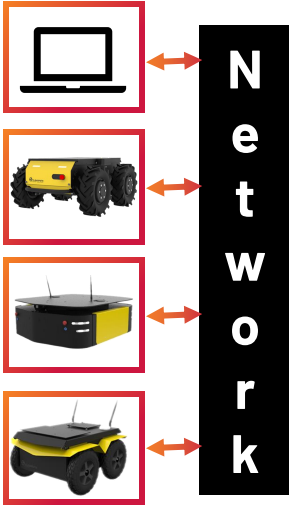
**Step 3:** Create each robot's profile. These are all the same:

```
<CycloneDDS>
  <Domain id="any">
    <General>
      <AllowMulticast>false</AllowMulticast>
    </General>
    <Discovery>
      <ParticipantIndex>auto</ParticipantIndex>
      <MaxAutoParticipantIndex>120</MaxAutoParticipantIndex>
      <Peers>
        <Peer address="localhost"/>
        <Peer address="observer"/>
      </Peers>
    </Discovery>
  </Domain>
</CycloneDDS>
```

- *docker/rmw\_configuration/lab1/cyclone/mock-robot-1.xml*
- *docker/rmw\_configuration/lab1/cyclone/mock-robot-2.xml*
- *docker/rmw\_configuration/lab1/cyclone/mock-robot-3.xml*



## | Exercise 4: Disable Multicast and Pin Peers



**Step 4:** Bring up the fleet but this time, pass in the RMW configuration directory we just created.

```
MOCK_RUN_PILOT=false scripts/workshop -t flat up 3 cyclone \  
  --model a300,j100,r100 \  
  --rmw-directory docker/rmw_configuration/lab1/cyclone
```

**Step 5:** Confirm that on any given robot, other robot topics are not discoverable

```
scripts/workshop shell mock-robot-1  
ros2 node list --no-daemon --spin-time 5      # only robot_1's  
exit
```

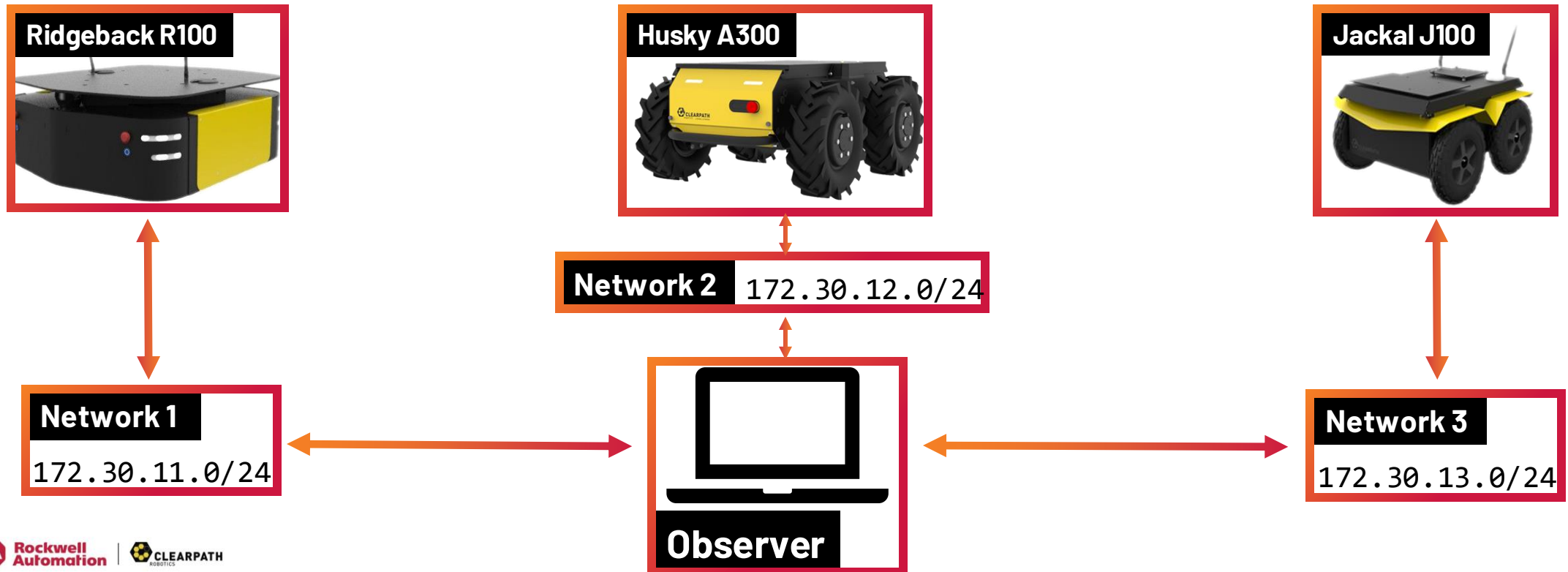
**Step 6:** On the observer, see if all robot's are discoverable

```
scripts/workshop shell observer  
ros2 node list --no-daemon --spin-time 5  # robot 1, 2, 3  
exit
```



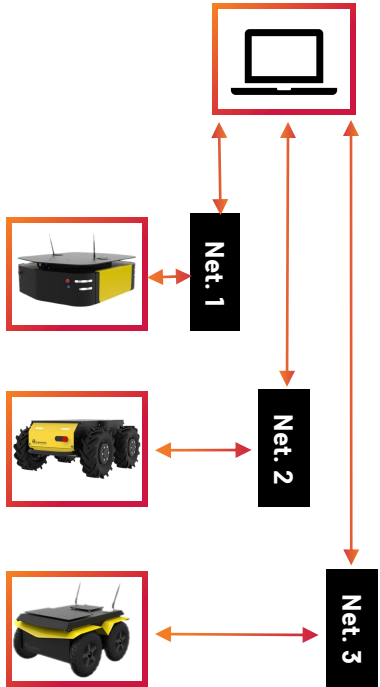
## | Exercise 5: Configure the RMW for the star

Domains provide a means to isolate discovery, but it does not prevent multicast discovery traffic. Instead, we can disable multicast and fix peers. Open the **Exercise 5: Configure the RMW for the star** (*5\_configure\_the\_rmw\_for\_the\_star.md*)





## | Exercise 5: Configure the RMW for the star



**Before we begin:** Bring up the fleet but this time, with the star topology

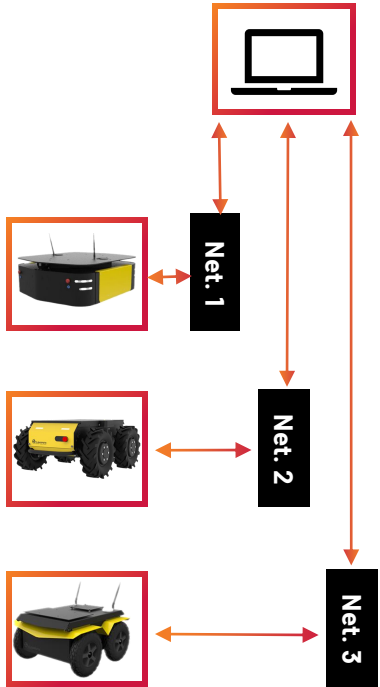
```
MOCK_RUN_PILOT=false scripts/workshop -t star up 3 cyclone
scripts/workshop shell observer
ros2 node list --no-daemon --spin-time 5
exit
```

The Cyclone DDS struggles with multi-interface setups. It excels when only one interface is required. You can check the observer configuration. All interfaces are specified and so are endpoints.

```
<Interfaces>
  <NetworkInterface address="172.30.11.20"/>
  <NetworkInterface address="172.30.12.20"/>
  <NetworkInterface address="172.30.13.20"/>
</Interfaces>
```



# | Exercise 5: Configure the RMW for the star



## Bring down the containers:

```
scripts/workshop -t star down
```

**Step 1:** Now, we will switch to using Fast DDS with the star topology.

```
MOCK_RUN_PILOT=false scripts/workshop -t star up 3 fastdds
```

## Step 2: Look at the generated observer configuration

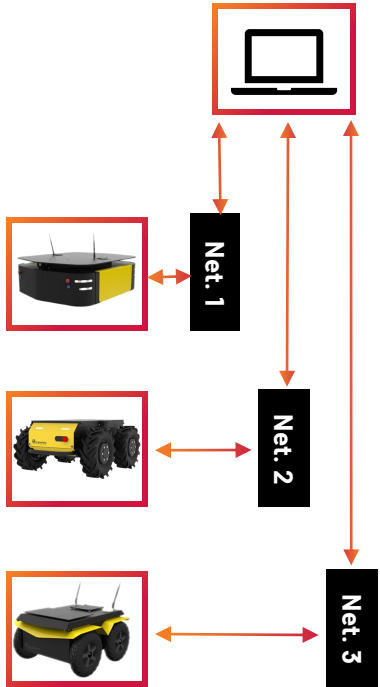
```
<interfaceWhiteList>
  <address>172.30.11.20</address>    <!-- observer's leg on robot 1's spoke -->
  <address>172.30.12.20</address>    <!-- observer's leg on robot 2's spoke -->
  <address>172.30.13.20</address>    <!-- observer's leg on robot 3's spoke -->
</interfaceWhiteList>
```

Each interface for each network is explicitly defined. Confirm the observer sees all robots.

```
scripts/workshop shell observer
ros2 node list --no-daemon --spin-time 5      # robot_1, robot_2, robot_3
```



## | Exercise 5: Configure the RMW for the star



**Step 3:** Isolate a single robot by trimming the interface white list. Edit the *docker/rmw\_configuration/star/fast/observer.xml*

```
<interfaceWhiteList>  
  <address>172.30.11.20</address>  
</interfaceWhiteList>
```

While in the observer shell:

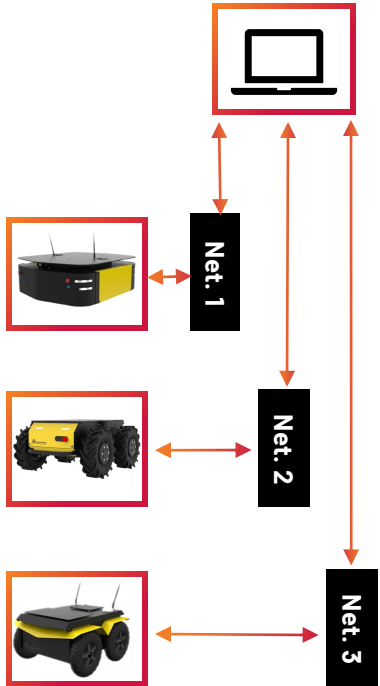
```
ros2 node list --no-daemon --spin-time 5          # only robot_1
```

Continue editing the *docker/rmw\_configuration/star/fast/observer.xml* by adding back one interface to the whitelist at a time until all of the robot nodes are discoverable.

```
<interfaceWhiteList>  
  <address>172.30.11.20</address>  
  <address>172.30.12.20</address>  
  <address>172.30.13.20</address>  
</interfaceWhiteList>
```



## | Exercise 5: Configure the RMW for the star



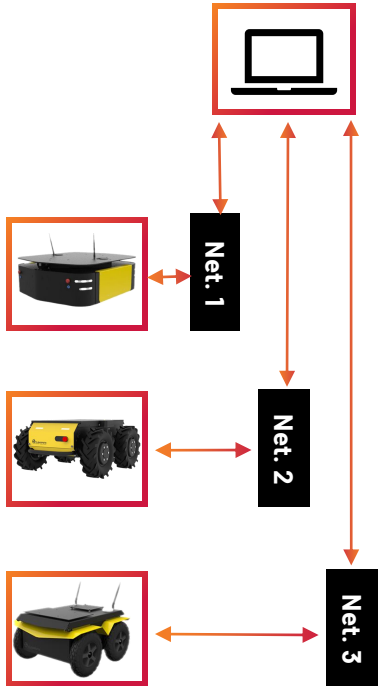
Now, we will disable multi-cast off on the Fast DDS configuration files.

**Step 1&2:** Edit the *docker/rmw\_configuration/star/fast/observer.xml* to include the **UnicastLocatorList**, **InitialPeers**, and **InitialPeersRange**

```
...
</interfaceWhiteList>
<maxInitialPeersRange>30</maxInitialPeersRange>
</transport_descriptor>
...
</discovery_config>
<metatrafficUnicastLocatorList>
  <locator><udp4><address>172.30.11.20</address></udp4></locator>
  <locator><udp4><address>172.30.12.20</address></udp4></locator>
  <locator><udp4><address>172.30.13.20</address></udp4></locator>
</metatrafficUnicastLocatorList>
<initialPeersList>
  <locator><udp4><address>172.30.11.11</address></udp4></locator>
  <locator><udp4><address>172.30.12.12</address></udp4></locator>
  <locator><udp4><address>172.30.13.13</address></udp4></locator>
</initialPeersList>
</builtin>
```



# Exercise 5: Configure the RMW for the star

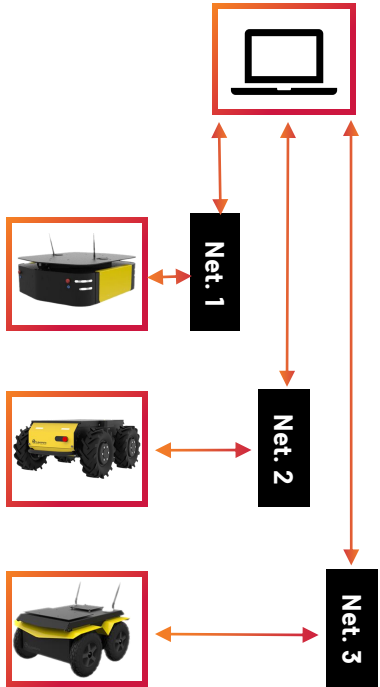


**Step 3:** Edit each of the mock robot configurations, in *docker/rmw\_configuration/star/fast/mock-robot-**<k>**.xml* to include the **UnicastLocatorList**, **InitialPeers**, and **InitialPeersRange**

```
...
  </interfaceWhiteList>
  <maxInitialPeersRange>30</maxInitialPeersRange>
</transport_descriptor>
...
</discovery_config>
<metatrafficUnicastLocatorList>
  <locator><udp4><address>172.30.11.11</address></udp4></locator>
</metatrafficUnicastLocatorList>
<initialPeersList>
  <locator><udp4><address>172.30.11.11</address></udp4></locator>
  <locator><udp4><address>172.30.11.20</address></udp4></locator>
</initialPeersList>
</builtin>
```



# Exercise 5: Configure the RMW for the star

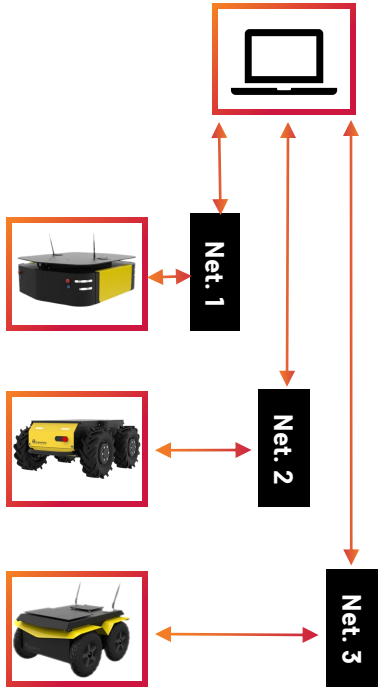


**Step 3:** Edit each of the mock robot configurations, in *docker/rmw\_configuration/star/fast/mock-robot-**<k>**.xml* to include the **UnicastLocatorList**, **InitialPeers**, and **InitialPeersRange**

```
...
  </interfaceWhiteList>
  <maxInitialPeersRange>30</maxInitialPeersRange>
</transport_descriptor>
...
</discovery_config>
<metatrafficUnicastLocatorList>
  <locator><udp4><address>172.30.11.11</address></udp4></locator>
</metatrafficUnicastLocatorList>
<initialPeersList>
  <locator><udp4><address>172.30.11.11</address></udp4></locator>
  <locator><udp4><address>172.30.11.20</address></udp4></locator>
</initialPeersList>
</builtin>
```



## | Exercise 5: Configure the RMW for the star



**Step 4:** Edit the container compose file *docker/compose/star.yml* to set the automatic discovery range to **SYSTEM\_DEFAULT** to prevent the environment variable from overriding the config.

```
ROS_AUTOMATIC_DISCOVERY_RANGE: SYSTEM_DEFAULT
```

**Step 5:** Bring the containers down, then bring it back up using the **--skip-gen** flag:

```
scripts/workshop -t star down  
MOCK_RUN_PILOT=false scripts/workshop -t star up 3 fastdds --skip-gen
```

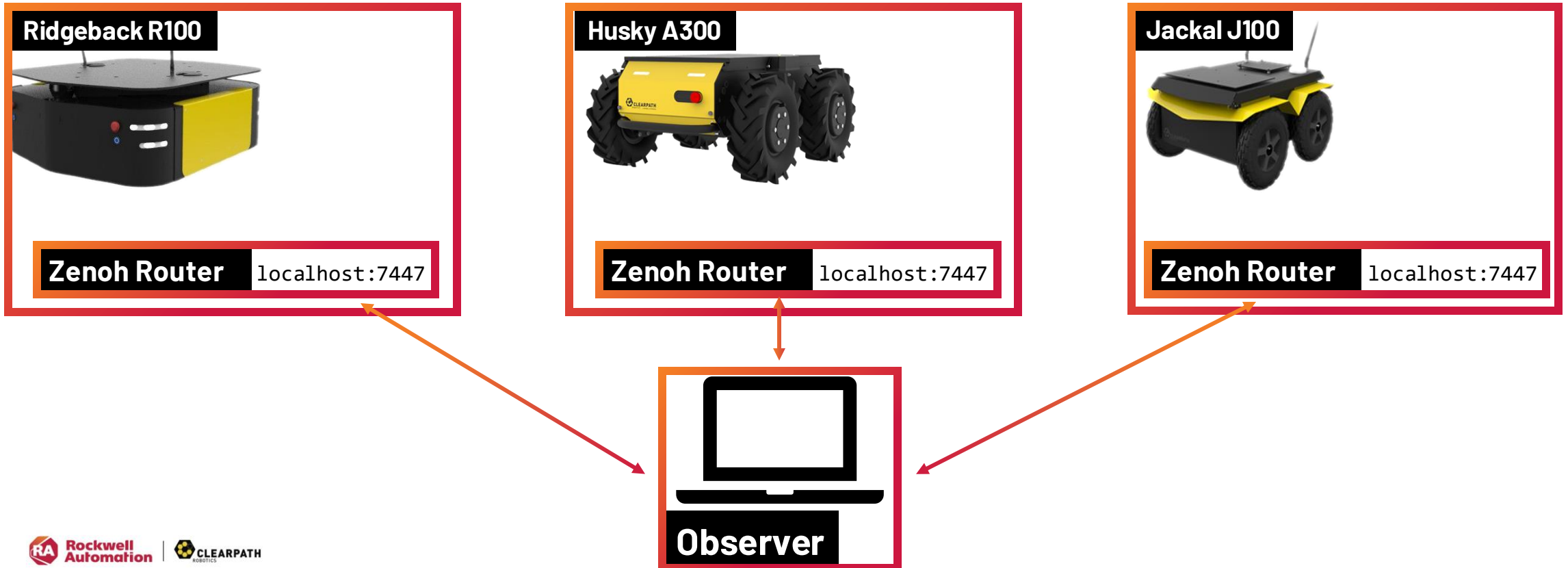
**Step 6:** Confirm the topics still show up without multicast

```
scripts/workshop shell observer  
ros2 node list --no-daemon --spin-time 5 # robot_1, robot_2, robot_3  
ros2 topic hz /robot_1/scan  
exit  
scripts/workshop -t star down
```



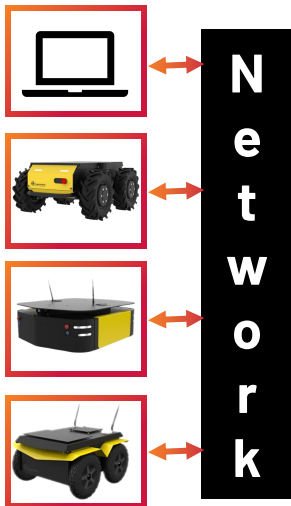
# | Exercise 6: Configure Zenoh

We move away from DDS and try the router based Zenoh.  
Open the **Exercise 6: Configure Zenoh on the flat bus**  
(*6\_configure\_zenoh.md*)





# Exercise 6: Configure Zenoh on Flat Topology



**Before we begin:** Bring up **three** robots on a **flat** topology using **Zenoh**.

```
MOCK_RUN_PILOT=false scripts/workshop -t flat up 3 zenoh
```

**Step 1:** Take a look at the robot router

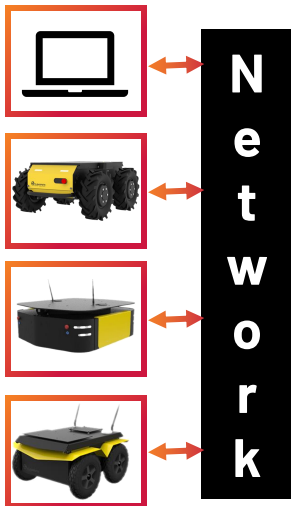
```
cat docker/rmw_configuration/flat/zenoh/mock-robot-router.json5
```

You will see a router configuration, where it listens on port **7447** on all interfaces and multicast is enabled:

```
{
  mode: "router",
  listen: { endpoints: ["tcp/0.0.0.0:7447"] },
  scouting: {
    multicast: { enabled: true, autoconnect: { router: ["router", "peer"], peer: ["router", "peer"] } },
    gossip: { enabled: true, autoconnect: { router: ["router", "peer"], peer: ["router", "peer"] } },
  },
  transport: { shared_memory: { enabled: false } },
}
```



# | Exercise 6: Configure Zenoh on Flat Topology



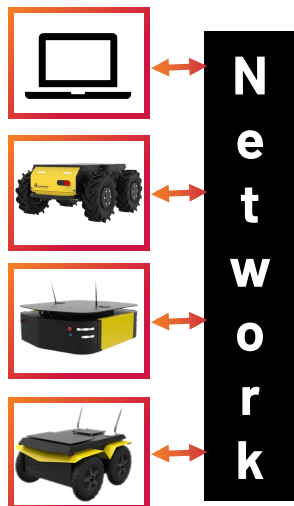
**Step 2:** The router configuration is not the same as the node configuration. Let's open the node configuration.

```
cat docker/rmw_configuration/flat/zenoh/mock-robot.json5
```

The node is setup as a "client" to the router. The router is set as the endpoint:

```
{  
  mode: "client",  
  connect: { endpoints: ["tcp/localhost:7447"] },  
  timestamping: { enabled: true },  
  transport: { shared_memory: { enabled: false } },  
}
```

## RA | Exercise 6: Configure Zenoh on Flat Topology



**Step 3:** The observer doesn't have a router. Instead, it must be set to find all the robots' routers.

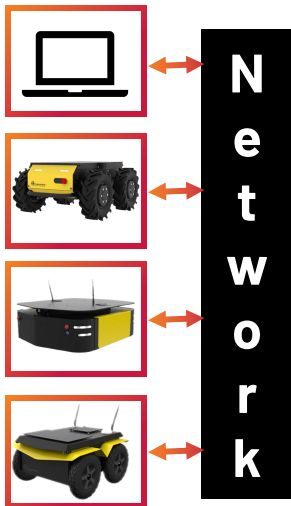
```
cat docker/rmw_configuration/flat/zenoh/observer.json5
```

The specific endpoints do not need to be set. Multicast handles discovery of routers (not individual participants).

```
{
  mode: "peer",
  scouting: {
    multicast: { enabled: true, autoconnect: { router: ["router", "peer"], peer:
["router", "peer"] } },
    gossip: { enabled: true, autoconnect: { router: ["router", "peer"], peer:
["router", "peer"] } },
  },
  transport: { shared_memory: { enabled: false } },
}
```



# | Exercise 6: Configure Zenoh on Flat Topology



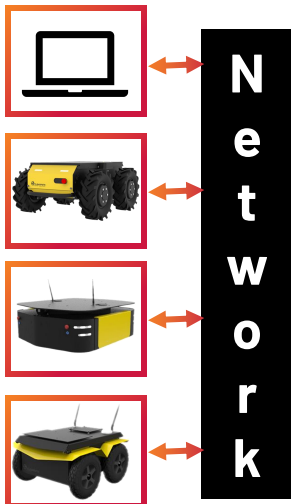
**Step 4:** Shell into the observer and make sure it can see all the robots' nodes.

```
scripts/workshop shell observer
ros2 topic list          # /robot_1/..., /robot_2/..., /robot_3/...
ros2 node list
exit
```

**Does it make sense to have a router on each robot?**



# | Exercise 6: Configure Zenoh using Downsampling

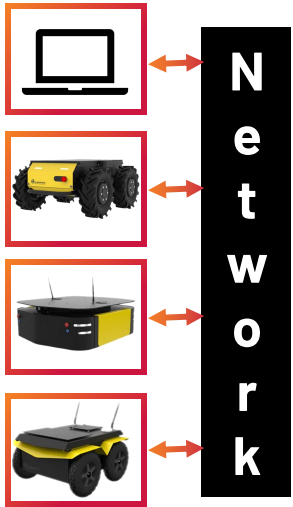


Since all traffic is routed through the router, we can make use of Zenoh's downsampling configuration. **Step 1:** Edit *docker/rmw\_configuration/flat/zenoh/mock-robot-router.json5*

```
{
  mode: "router",
  listen: { endpoints: ["tcp/0.0.0.0:7447"] },
  scouting: {
    multicast: { enabled: true, autoconnect: { router: ["router", "peer"], peer: ["router", "peer"] } },
    gossip: { enabled: true, autoconnect: { router: ["router", "peer"], peer: ["router", "peer"] } },
  },
  downsampling: [
    {
      id: "image_1hz_wire",
      interfaces: ["eth0"], // only the NIC toward the observer; lo is excluded
      flows: ["egress"],
      messages: ["put"],
      rules: [
        { key_expr: "**/camera/image_raw/**", freq: 1.0 },
      ],
    },
  ],
  transport: { shared_memory: { enabled: false } },
}
```



# | Exercise 6: Configure Zenoh using Downsampling



**Step 2:** Bring down the containers and bring them back up with the **--skip-gen** flag

```
scripts/workshop -t flat down  
MOCK_RUN_PILOT=false scripts/workshop -t flat up 3 zenoh --skip-gen
```

**Step 3:** Compare the camera rate on the robot versus the observer

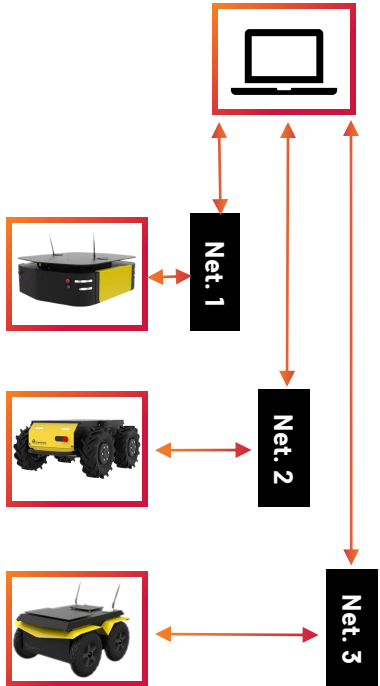
```
scripts/workshop shell mock-robot-1  
ros2 topic hz /robot_1/sensor_0/camera/image_raw    # full sensor rate  
exit
```

On the robot, the rate is unchanged. Now check the observer

```
scripts/workshop shell observer  
ros2 topic hz /robot_1/sensor_0/camera/image_raw    # ~1.0 Hz;  
exit
```



# Exercise 6: Configure Zenoh on Star Topology



**Step 1:** Bring down the fleet. Then bring up again, but this time, with the star topology

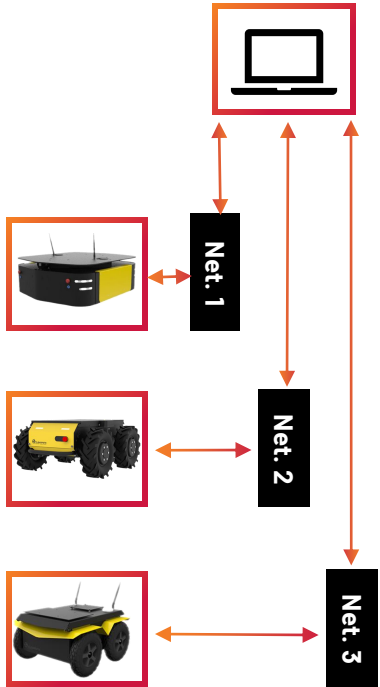
```
scripts/workshop down  
MOCK_RUN_PILOT=false scripts/workshop -t star up 3 zenoh
```

**Step 2:** Each robot still has it's own router. Open the observer configuration to see how it sets up endpoints:

```
{  
  mode: "peer",  
  scouting: {  
    multicast: { enabled: false, ... }, // no shared segment: nothing is scouted  
    gossip:    { enabled: true,  ... },  
  },  
  connect: { endpoints: [  
    "tcp/172.30.11.11:7447", // mock-robot-1's router  
    "tcp/172.30.12.12:7447", // mock-robot-2's router  
    "tcp/172.30.13.13:7447", // mock-robot-3's router  
  ] },  
  listen: { endpoints: [ // the observer's own leg on each spoke  
    "tcp/172.30.11.20:7447",  
    "tcp/172.30.12.20:7447",  
    "tcp/172.30.13.20:7447",  
  ] },  
  transport: { shared_memory: { enabled: false } },  
}
```



# Exercise 6: Configure Zenoh on Star Topology



**Step 3:** Now modify the observer's config to only see a single robot.

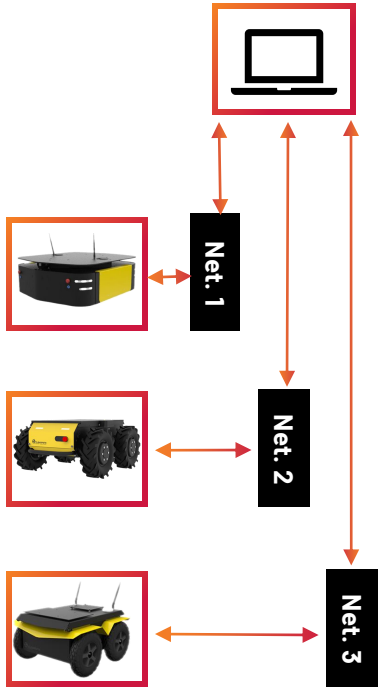
```
{
  mode: "peer",
  scouting: {
    multicast: { enabled: false, ... },    // no shared segment: nothing is scouted
    gossip:    { enabled: true,  ... },
  },
  connect: { endpoints: [
    "tcp/172.30.11.11:7447",    // mock-robot-1's router
  ] },
  listen: { endpoints: [      // the observer's own leg on each spoke
    "tcp/172.30.11.20:7447",
    "tcp/172.30.12.20:7447",
    "tcp/172.30.13.20:7447",
  ] },
  transport: { shared_memory: { enabled: false } },
}
```

Then, open the observer and see what nodes are listed:

```
scripts/workshop shell observer
ros2 node list --no-daemon --spin-time 5      # only robot_1
```



# Exercise 6: Configure Zenoh on Star Topology



**Step 4:** Now modify the observer's config to only see the first and second robots.

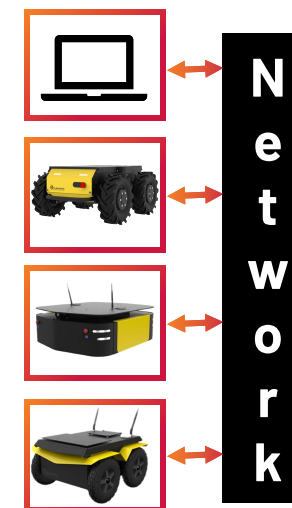
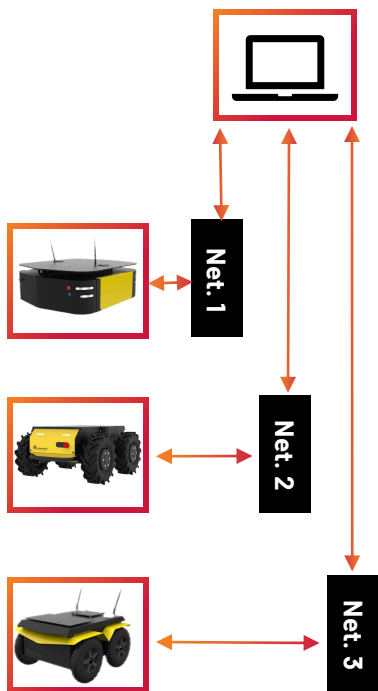
```
{
  mode: "peer",
  scouting: {
    multicast: { enabled: false, ... },    // no shared segment: nothing is scouted
    gossip:    { enabled: true,  ... },
  },
  connect: { endpoints: [
    "tcp/172.30.11.11:7447",    // mock-robot-1's router
    "tcp/172.30.12.12:7447",    // mock-robot-2's router
  ] },
  listen: { endpoints: [          // the observer's own leg on each spoke
    "tcp/172.30.11.20:7447",
    "tcp/172.30.12.20:7447",
    "tcp/172.30.13.20:7447",
  ] },
  transport: { shared_memory: { enabled: false } },
}
```

# RA | Lab 1: Conclusion

We explored:

- The workshop script and familiarized ourselves with the commands,
- The flat and star topologies,
- The configuration files for the three middleware configurations, and
- We explored multi-cast versus setting a predefined endpoint list

But, **what is actually happening** on the wire?  
**How can we tell what we need to configure** when something is just not working?





# Lab 2: Diagnostic tools



## | Lab 2 objectives

- Use Webshark to look for network problems such as retries
- Use netem to emulate network problems
- Force retransmission of SEDP information to identify topics



# | Terminology (refresher?)

SPDP - Simple Participant Discovery Protocol (DDS)

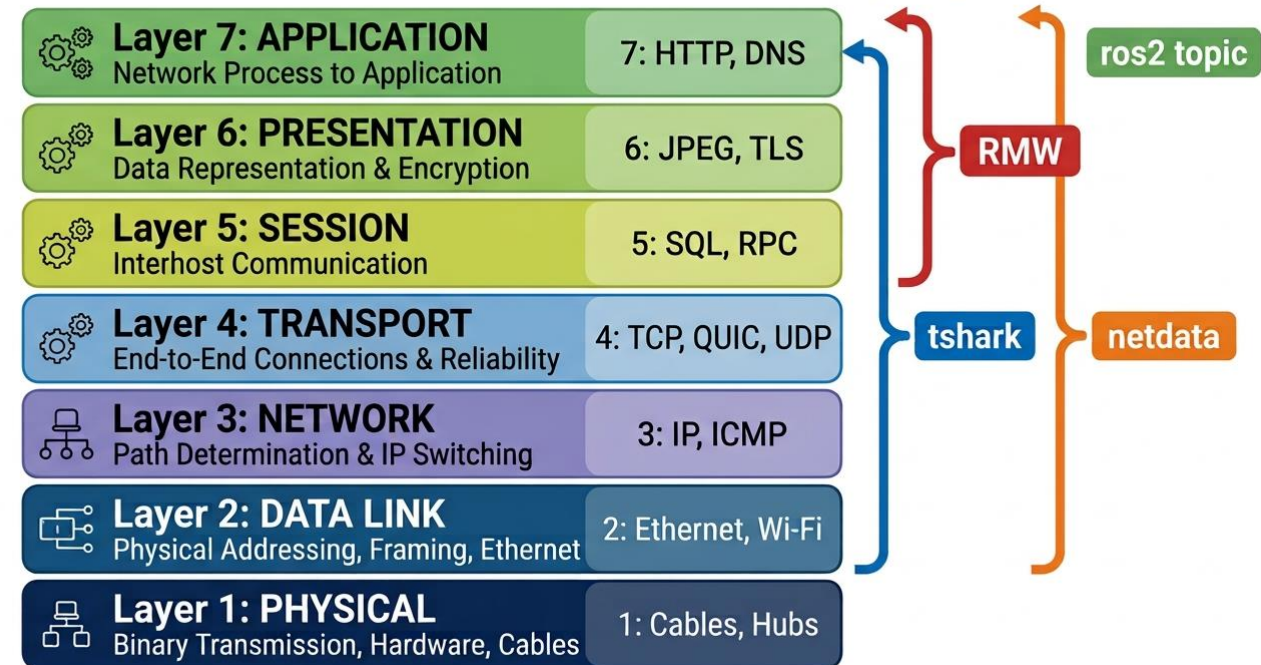
- o automatically find other robots, computers, or software nodes on a network

SEDP - Simple Endpoint Discovery Protocol (DDS)

- o topics, data types, and Quality of Service (QoS) configurations of all publishers (DataWriters) and subscribers

RTSP - Real-Time Streaming Protocol (IETF)

## OPEN SYSTEMS INTERCONNECTION (OSI) MODEL

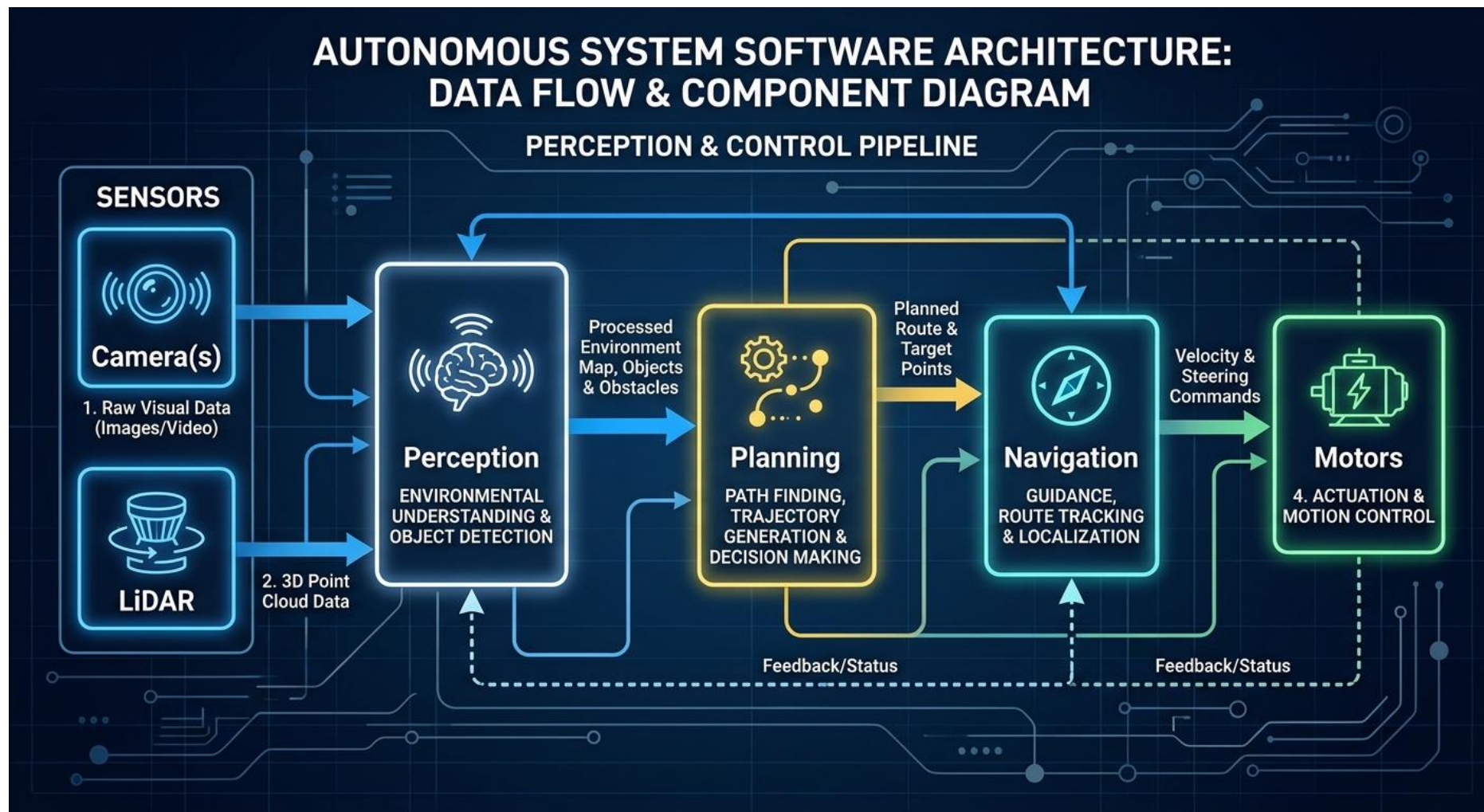




# | Running ROS on a single system is easy...

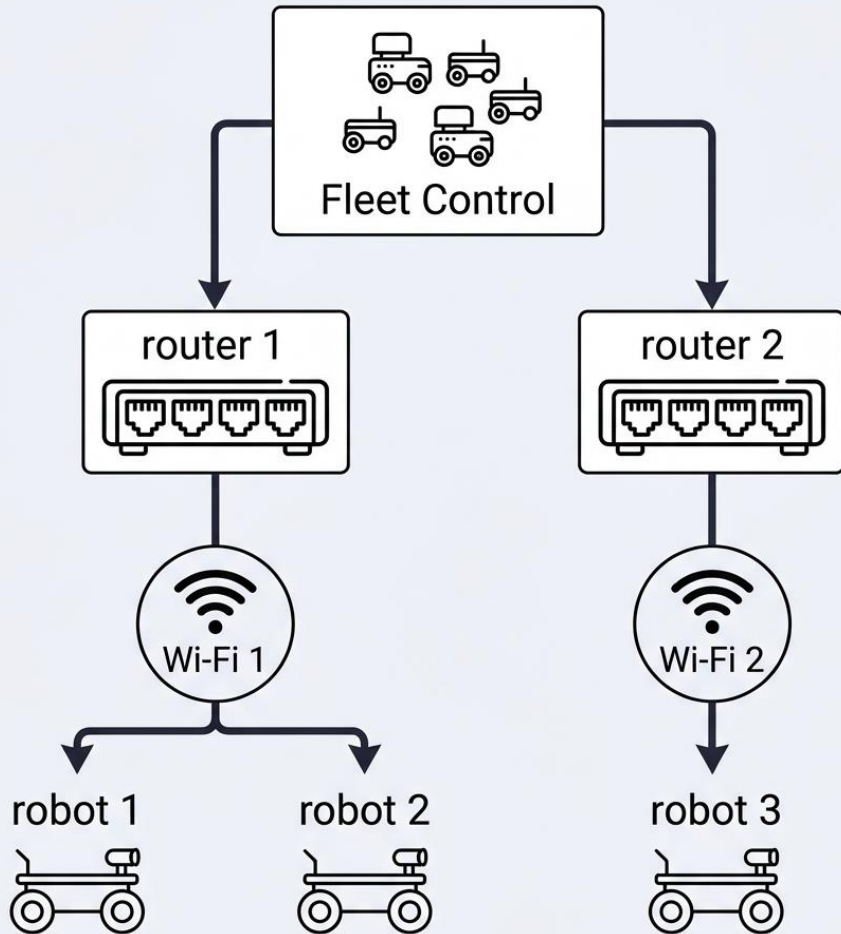
Comms

- High reliability
- High bandwidth





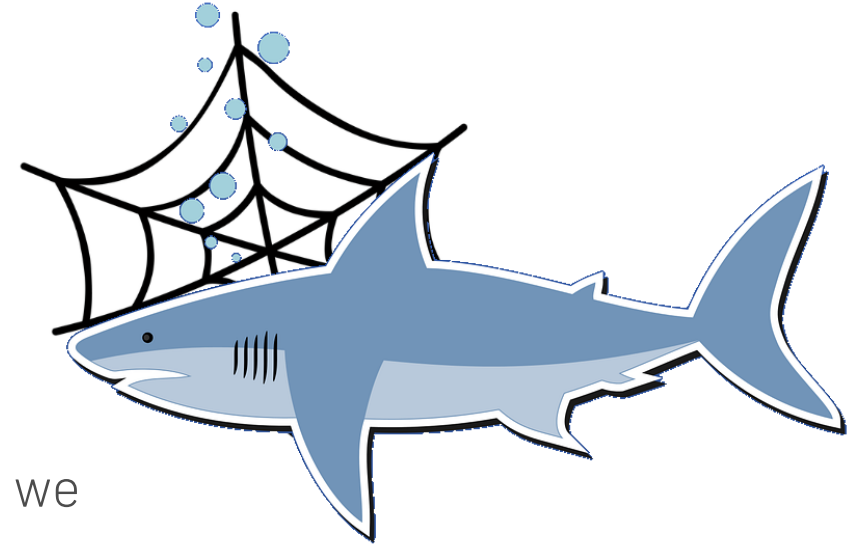
# | ... but a larger scale deployment has challenges



- RF interference
- Wireless coverage and quality
- Multicast not allowed
- Multicast discovery storms
- Encryption overhead
- MTU mismatch
- Firewall
- Bad router in network (failed or misconfigured)
- Buffering on routers, especially with varying link speeds



# | Webshark



- Like Wireshark, but with a web front-end (GPL-2 license)
- Easy containerized setup, avoiding version mismatches and we provide ROS 2 filters

Why not use MCAPs (rosbags)?

- These record the app data a node has seen. It does not capture network behaviour: dropped packets, corrupted packets, re-transmits, timing differences between sender and receiver, etc.



# | Lab 2 summary

3 robots

Each publishes:

- Image topic 41KB
- Lidar scan topic small, 0.8KB

Dimensions of exploration:

- RMW: Cyclone DDS, Fast DDS, Zenoh
- Netem profile: healthy, poor (*more in lab3*)



# | Lab 2 cheat sheet

- Link: <http://localhost:8085/webshark/>
- Commands:

```
scripts/workshop status
```

```
RMW=fast scripts/workshop -t flat up 3 --webshark
```

```
scripts/workshop capture 3 --files 12
```

```
...
```

```
scripts/workshop down
```

# | Lab 2 Exercise 1 Capturing a PCAP

Target time: 10min

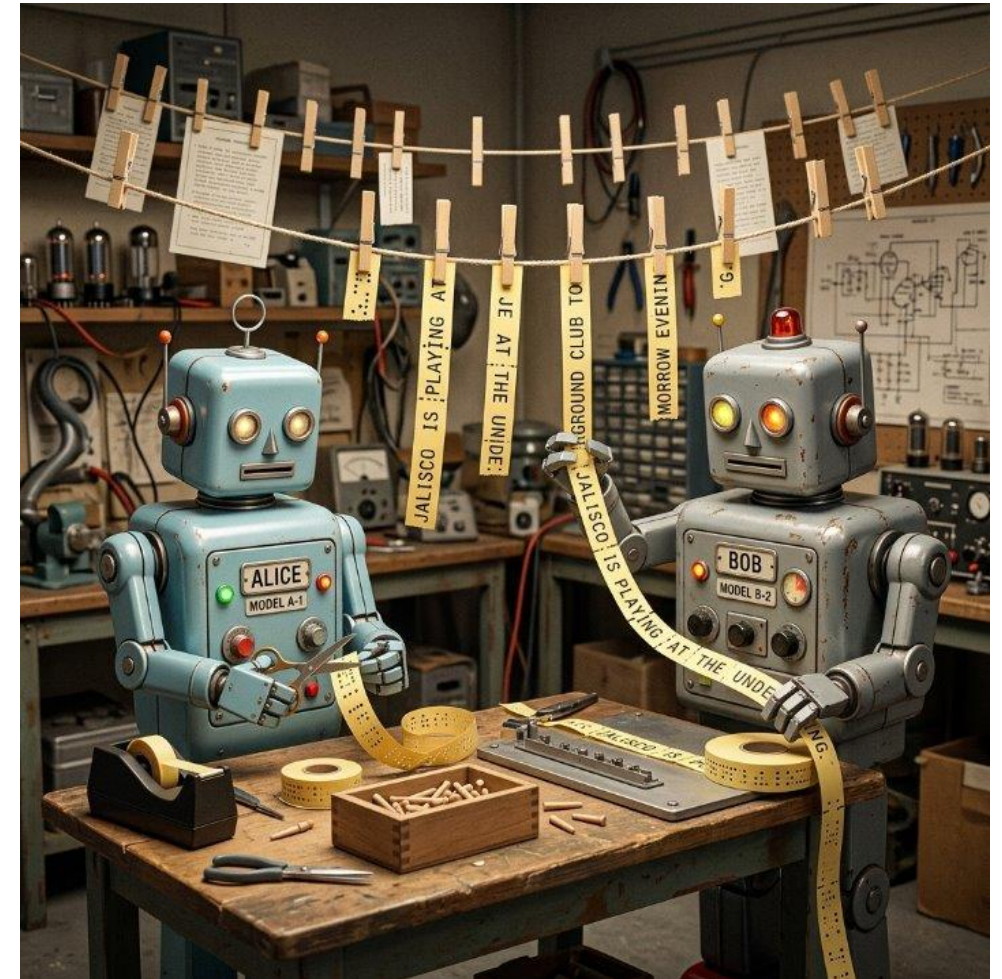
- Often a problem when you get captures from a customer, or even internally during longer ad-hoc testing: it is missing topic labels
- Take a PCAP
- Notice that you have no idea what the endpoints are...
- Force the endpoints to be reannounced



# RA | Lab 2 Exercise 2 Beyond the MCAPs

Target time: 8min

- Simulate a lossy network. Not really a "poor" quality one, really just a realistic wireless.
- Why do some ROS topics have trouble?
  - Notice the topic QoS setting: Reliability: BEST\_EFFORT



# | Lab 2 Exercise 2 Beyond the MCAPs

## Learnings

- Fast DDS hands a large datagram to IP to handle.
  - If a single IP fragment is lost, the receiving network stack discards all the other received fragments for that datagram and relies on upper-layer protocols to handle the loss.
  - Will still split datagrams larger than 65,500 bytes itself (configurable).
- Cyclone DDS splits large datagrams itself.
- Zenoh splits large datagrams itself.



# Lab 2 Exercise 3 Lies, true lies, and PCAPs

Target time: 8min

- Write down your Zenoh behaviour predictions
- Test out your predictions...





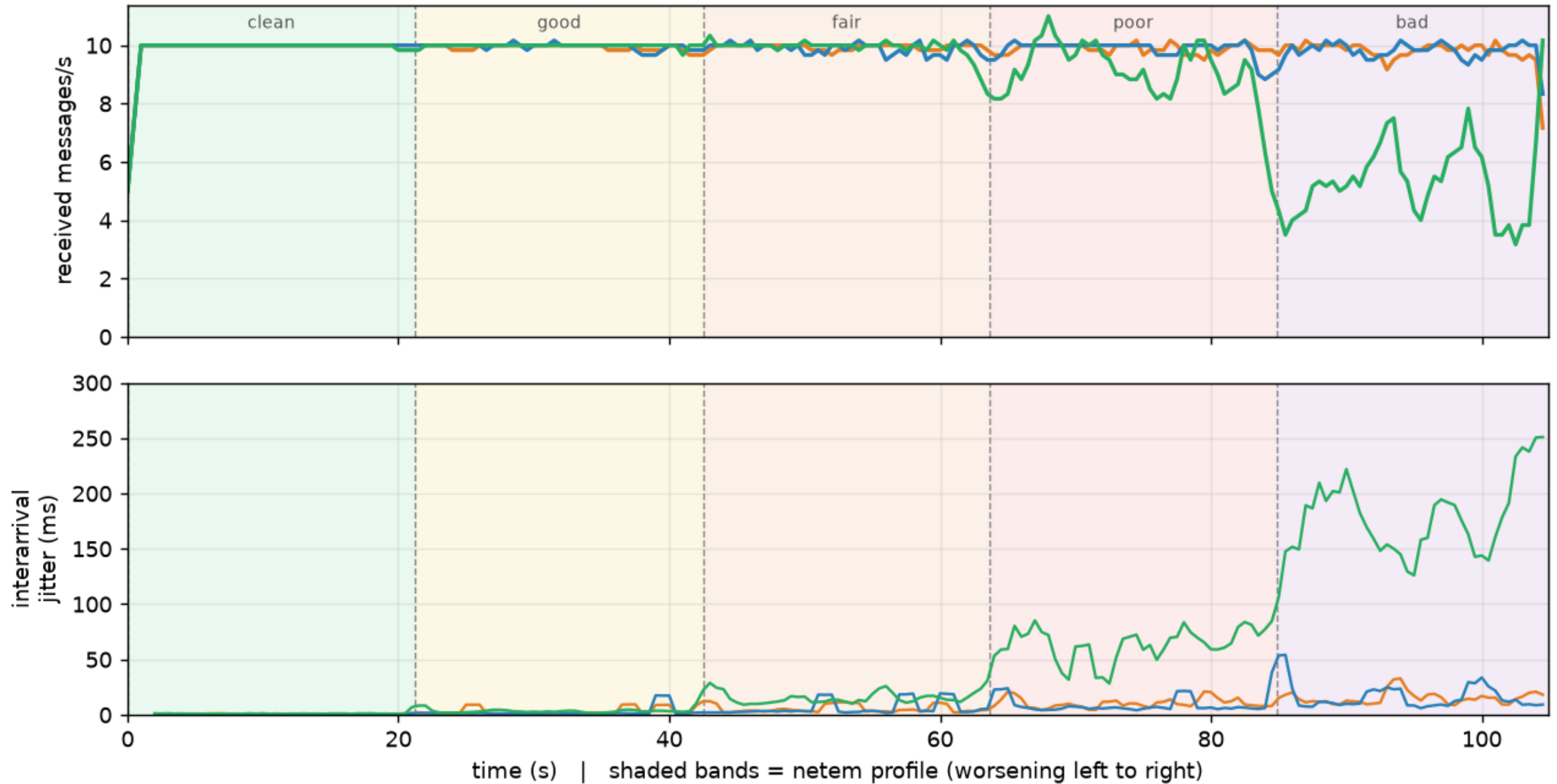
# | Lab 2 Exercise 3 Lies, true lies, and PCAPs

Learnings (Does Zenoh have a paradox?)

- When the impairment goes from bad to worse, the recovery actions matter
  - These can swamp the network
- Zenoh (as configured) is running a custom TCP implementation to handle the fragmentation and retransmits
- Even though BEST\_EFFORT is specified, Zenoh uses TCP retransmits.
- Check out more details in [lab2-on-the-wire/beyond-the-wire/](#)

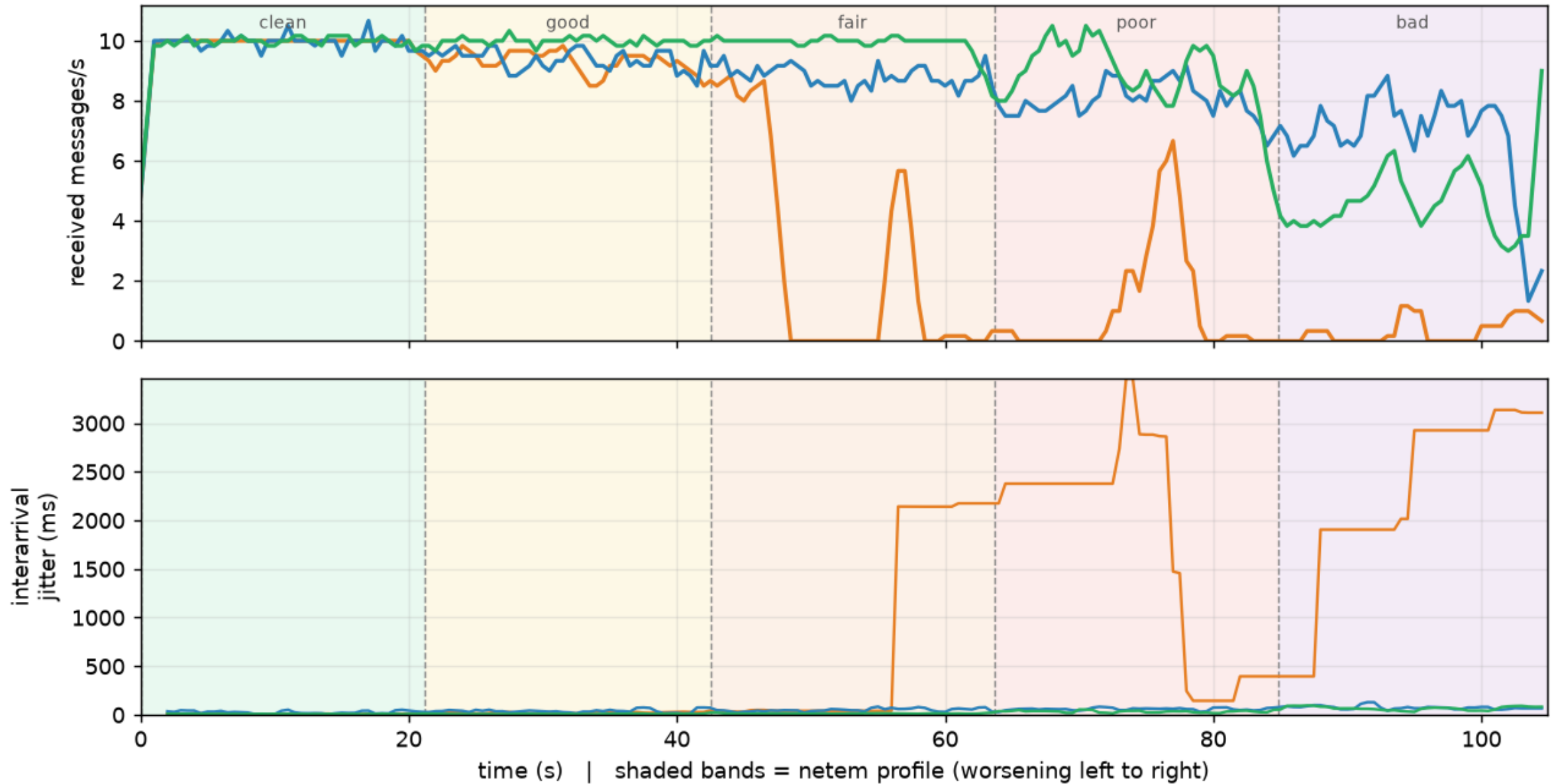
# Delivered messages by RMW - scan

Fast DDS   Cyclone DDS   Zenoh



## Delivered messages by RMW - camera

Fast DDS Cyclone DDS Zenoh





# | EXTRA Wireshark

- Common problems
- User can't see interfaces (lo, wl(p2s0f0), enx)
  - `sudo usermod -aG wireshark $USER`



# Lab 3: Stress Testing

# | Agenda

| Section                | Duration (minutes) |
|------------------------|--------------------|
| Intro                  | 2                  |
| Monitoring: Netdata    | 3                  |
| Exercise 1             | 10                 |
| Stress Testing         | 5                  |
| Exercise 2             | 10                 |
| Exercise 3             | 15                 |
| Transport Layer Impact | 5                  |
| Exercise 4             | 10                 |
| Wrap-up                |                    |



# | From the Lab to Production

- Previous labs taught you how to use low level tools to troubleshoot issues once an issue is detected
  - How do you detect issues? -> **monitoring** -> allows you to automate testing and alerting
  - How do you trigger issues before you ship/deploy? -> **stress testing**



## Objectives

1. Monitor deployment
- 
2. Simulate and stress deployment





# | Monitoring: Netdata

- Real-time monitoring for hosts, containers and applications — auto-discovers what's running and builds dashboards with zero configuration.
- 1s granularity keeps short spikes visible that Prometheus-style 15-60s scrapes smooth away.

## What it is

A **monitoring agent that collects, stores and visualises metrics** on the machine it runs on.

**Auto-detects services** and **builds dashboards** with no configuration.

**Built-in anomaly detection** and alarms surface issues before someone files a ticket.

## Deployment model

One lightweight **agent per node**, collecting locally.

Tiered **time-series database** on the node — no central DB needed to get started.

**Dashboard** served straight off the agent on **port 19999**: open it and you see live data.

Agents **can stream to a parent node** for fleet-wide retention and a single pane of glass.

## Data available immediately

**System**: CPU, memory, disk I/O, network interfaces, pressure stall information.

**Per-application groups** (apps.plugin): CPU, memory and I/O attributed to process families.

**Containers**, systemd services and common services via auto-detected collectors.



# Netdata: In Our Lab Exercises

- Stock charts answer some questions – custom signals needed where the Agent is blind.



## How we run it

Netdata as a constrained container in the workshop stack.

Shares host PID + network namespaces,

## Limitations

No host cgroup mount – no per-container CPU, memory, network or I/O accounting.

Container network interfaces and in-container TCP retransmits aren't visible to the host Agent.

A tradeoff of portability, not a Netdata limit

## What we added

Lab 3 AP – qdisc backlog, drops and profile from inside wifi-ap.

Lab 3 Fleet – RTPS discovery traffic, default vs pinned.

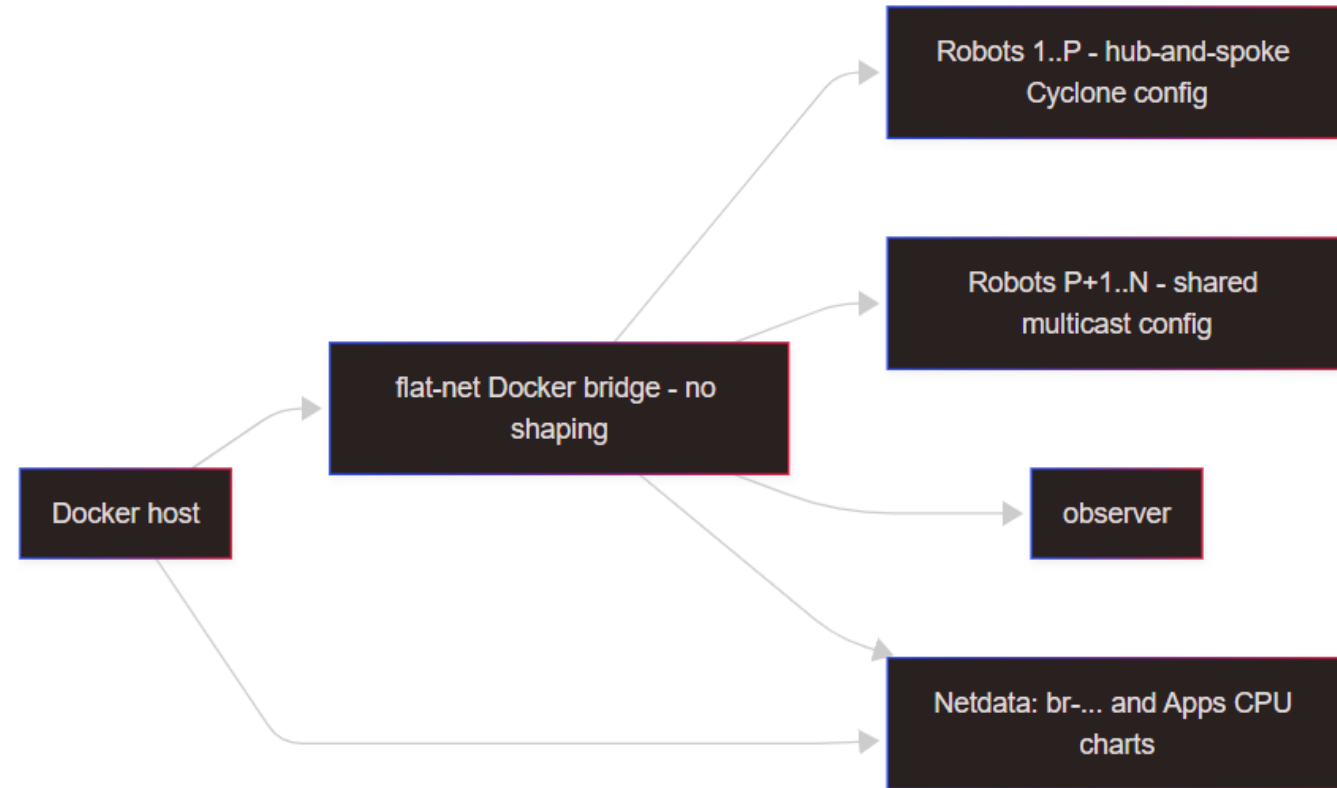
Both via the built-in StatsD collector (UDP 8125).



# Exercise 1: Take the Fleet's Pulse

Target time: 10 minutes

- **Goal:** Explore Netdata by bringing up a fleet
  - Default networking charts will have limited data
    - Custom lab3 charts focused on participant discovery
      - Use concepts from lab2(tcpdump) to capture and present missing data
  - Apps/CPU default charts
    - Custom app groups



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1\\_take\\_the\\_fleets\\_pulse.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1_take_the_fleets_pulse.md)

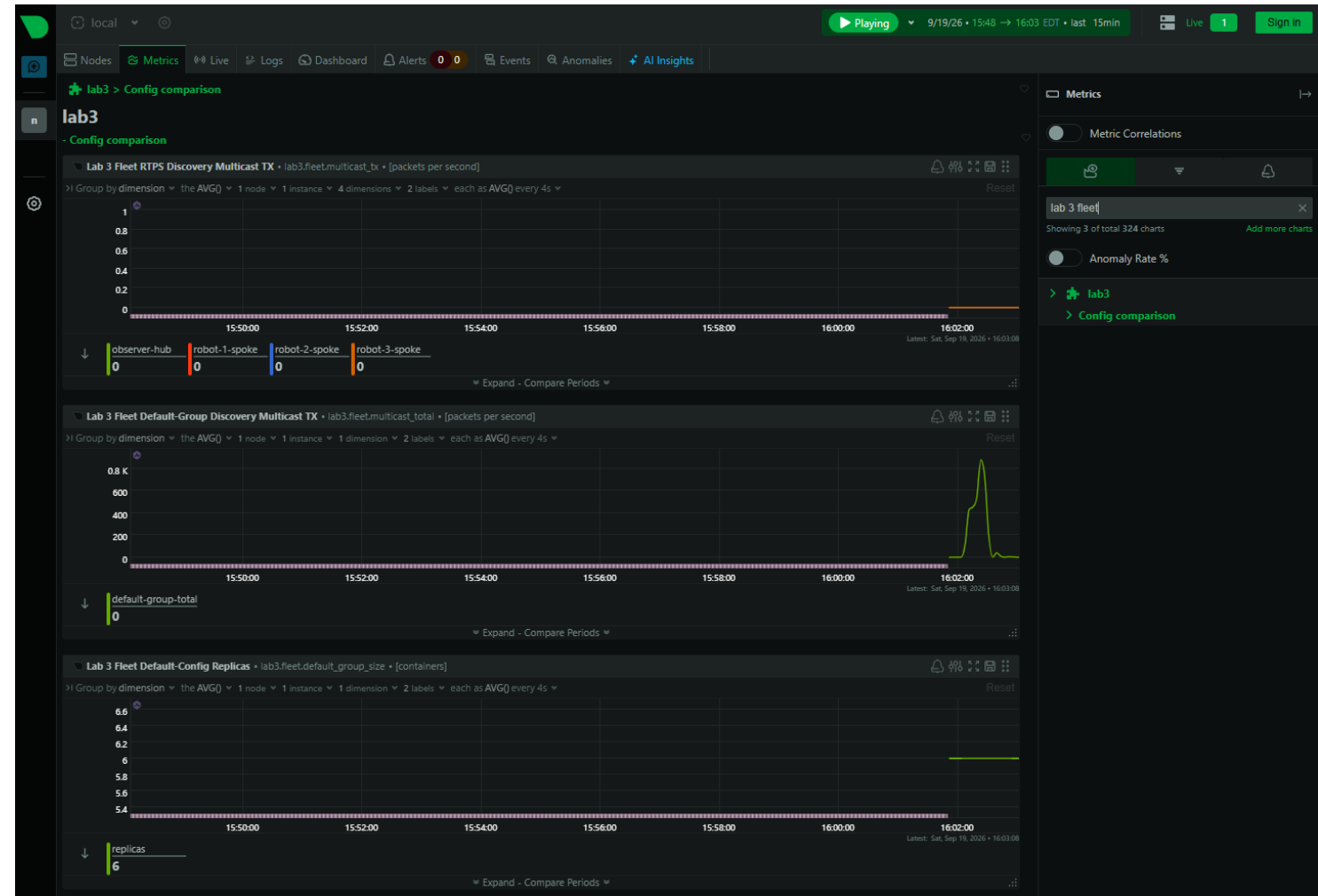


# Exercise 1: Take the Fleet's Pulse

## RTPS Discovery

- Discussion

- What's the difference between the default robots and the extra robots deployed?



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1\\_take\\_the\\_fleets\\_pulse.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1_take_the_fleets_pulse.md)

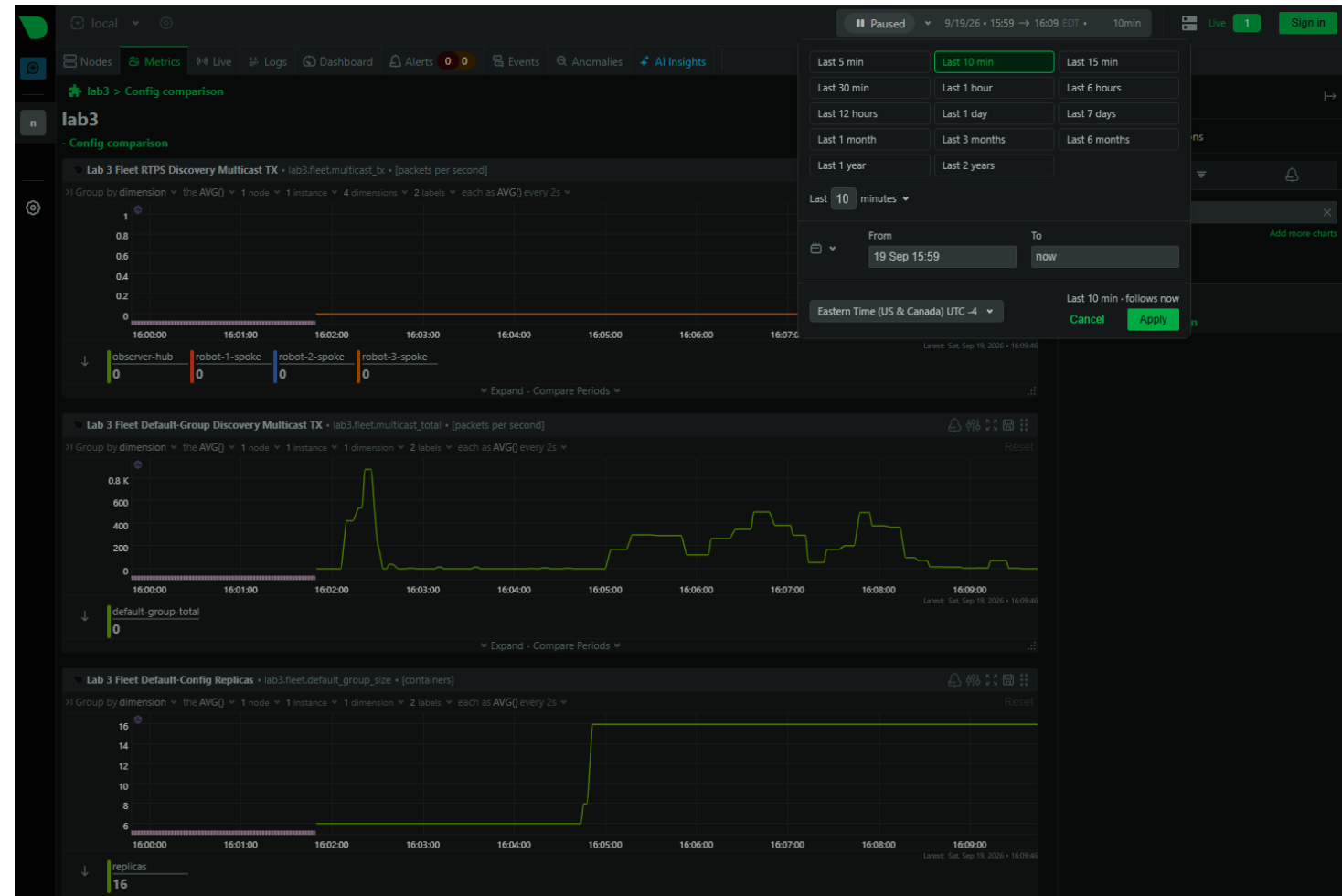


# Exercise 1: Take the Fleet's Pulse

## RTPS Discovery

- Discussion

- Behavior of bringing up 10 more robots
- What did you observe when using Zenoh?



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1\\_take\\_the\\_fleets\\_pulse.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1_take_the_fleets_pulse.md)

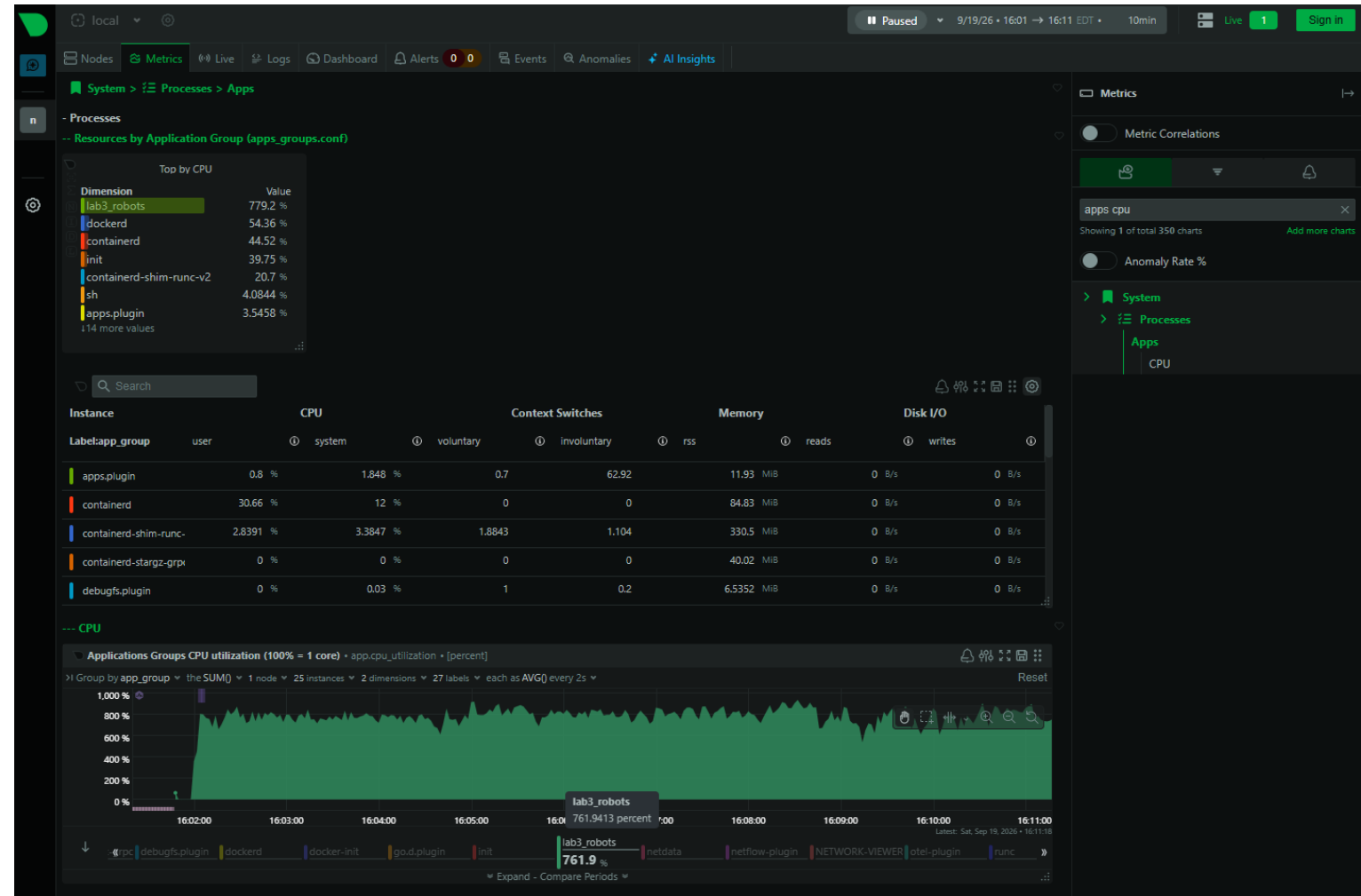


# Exercise 1: Take the Fleet's Pulse

## Default Netdata charts

- Discussion

- Any particularly interesting or useful Netdata graphs or stats to share?



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1\\_take\\_the\\_fleets\\_pulse.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/1_take_the_fleets_pulse.md)



# | Stress Testing: Simulate Real Deployments

- In-house testing rarely reproduces real deployment conditions – clean lab networks, few nodes, no interference.
- Close the gap early – finding it in the lab is far cheaper than finding it in the field.

## The gap

**Lab networks are clean**, small and predictable.

**Real deployments bring loss, jitter, contention** and many more nodes.

**Issues surface after deployment**, when changes are costly.

## Shape the network

**netem** to inject latency, jitter, loss and bandwidth limits.

A **shared AP queue** reproduces contention for the medium.

Test against the conditions the robot will actually meet.

## Scale the fleet

**Mock robots** to push node and discovery counts up.

**Replayed packet captures** for "realistic" traffic.

Find the breaking point of the deployment before the customer does.



# | tc-netem: Shaping the Medium Under Your Stack

Inject delay, jitter and loss on purpose

- sch\_netem is a Linux kernel queueing discipline that injects delay, jitter, loss, duplication, corruption and reordering into an egress queue.
- Configured with tc qdisc on any interface — physical NIC, veth, bridge, or an ifb redirect for ingress shaping.

## What it is

A kernel **qdisc**, not a proxy — **impairment** happens **below the DDS/Zenoh stack**.

Configured through **tc**

**Works on any interface:** NIC, veth, bridge, or ifb.

## Where you'd use it

Bench-reproducing a "bad Wi-Fi day" (modelling a shared medium).

Modelling an outdoor link with only a cheap AP.

On a physical robot? Possible, not preferred.

## How to think about it

**tc (traffic control)** installs qdiscs, classes and filters on an iface

**qdisc (queueing discipline):** algorithm that decides what to do with packets on the network iface

**netem:** applies qdisc impairment

**htb (hierarchical token bucket):** rate limits



# | tc-netem: Profiles, Examples and Gotchas

The knobs you'll actually turn in Lab 3

- The Lab 3 profile ladder – `scripts/workshop -t routed netem <profile>` – you'll refer back to during Exercises 2 and 3.

| Profile  | Rate     | Delay         | Loss (GE) |
|----------|----------|---------------|-----------|
| good     | 100 Mbit | 5 ms ± 1 ms   | 0.1 %     |
| degraded | 100 Mbit | 15 ms ± 5 ms  | –         |
| lossy    | 50 Mbit  | 30 ms ± 10 ms | 5 %       |
| bad      | 20 Mbit  | 80 ms ± 30 ms | 15 %      |

## Minimal examples

```
tc qdisc add dev eth0 root netem delay 30ms 10ms loss 5%
tc qdisc add dev ifb0 root htb rate 50mbit → netem → sfq
tc qdisc change dev eth0 root netem loss gmodel 5% 40%
tc qdisc del dev eth0 root
```

## Gotchas

netem **shapes egress only** – use ifb + mirrored to shape “incoming” traffic.

**Stacking errors:** apply the same profile in two places along one path (robot egress and in-path bridge) and the impairments compound (verify with `tc qdisc show`)



# Deployment Topology

Exercises 2 - 4



# | Exercise 2: Watch a Healthy Fleet Fail

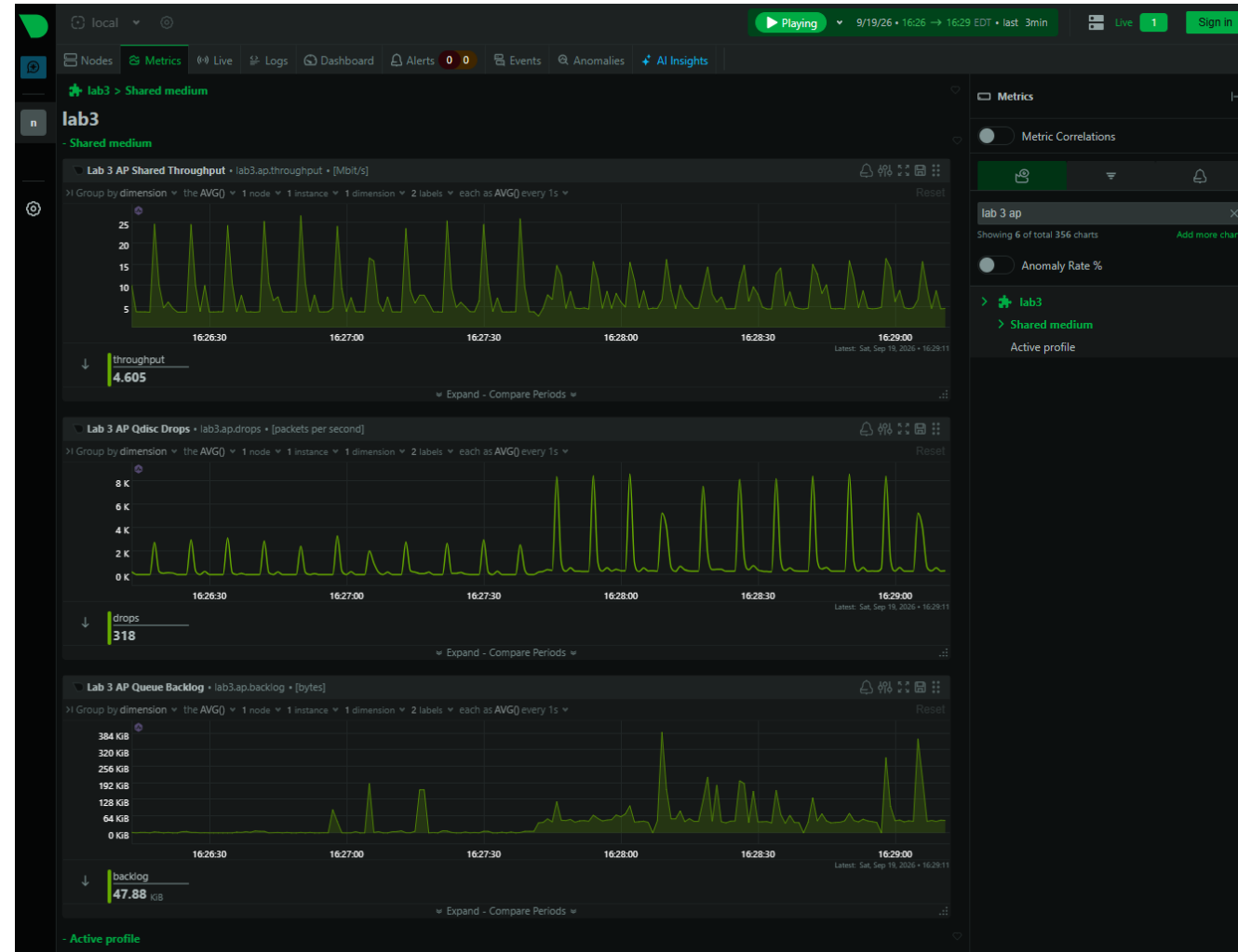
Target time: 10 minutes

- **Goal:** deploy shared-medium netem emulator and apply traffic shaping
  - Focus here is more on traffic shaping as scaling will hit your host's resource limits
  - Use a "Fleet Manager" display to monitor fleet health
  - Use new Netdata custom charts to track network impairments
  - What causes ROS 2 data to cross the network
  - How does "bad network" manifest? What are the symptoms?

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2\\_watch\\_a\\_healthy\\_link\\_fail.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2_watch_a_healthy_link_fail.md)

# Exercise 2: Watch a Healthy Fleet Fail

- Discussion
  - New Netdata charts
    - Shared medium
    - Active *netem* profile

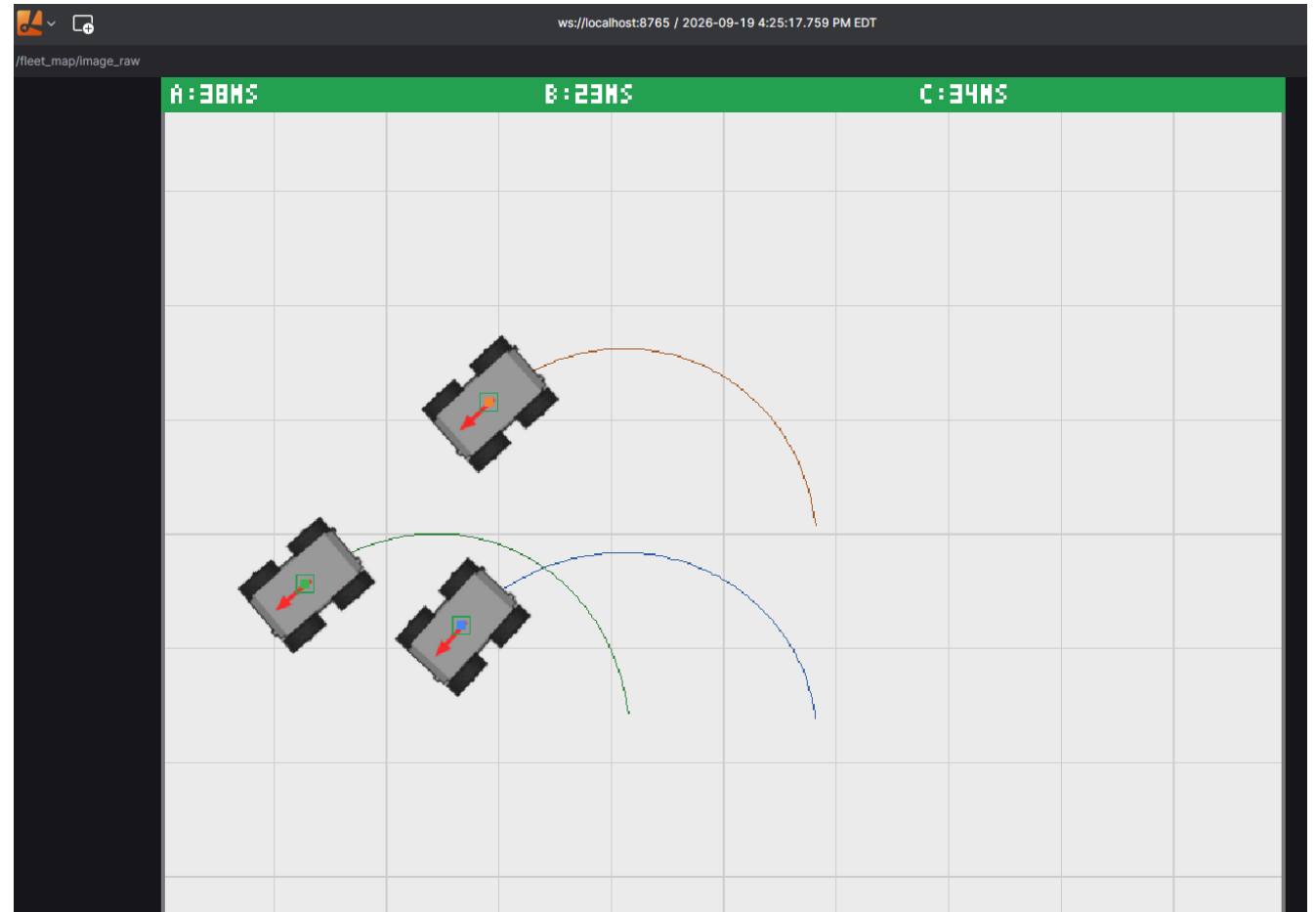


[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2\\_watch\\_a\\_healthy\\_link\\_fail.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2_watch_a_healthy_link_fail.md)

# RA | Exercise 2: Watch a Healthy Fleet Fail

- Discussion

- Healthy fleet
- When does ROS 2 topic data cross the AP (i.e. when is network data generated)?

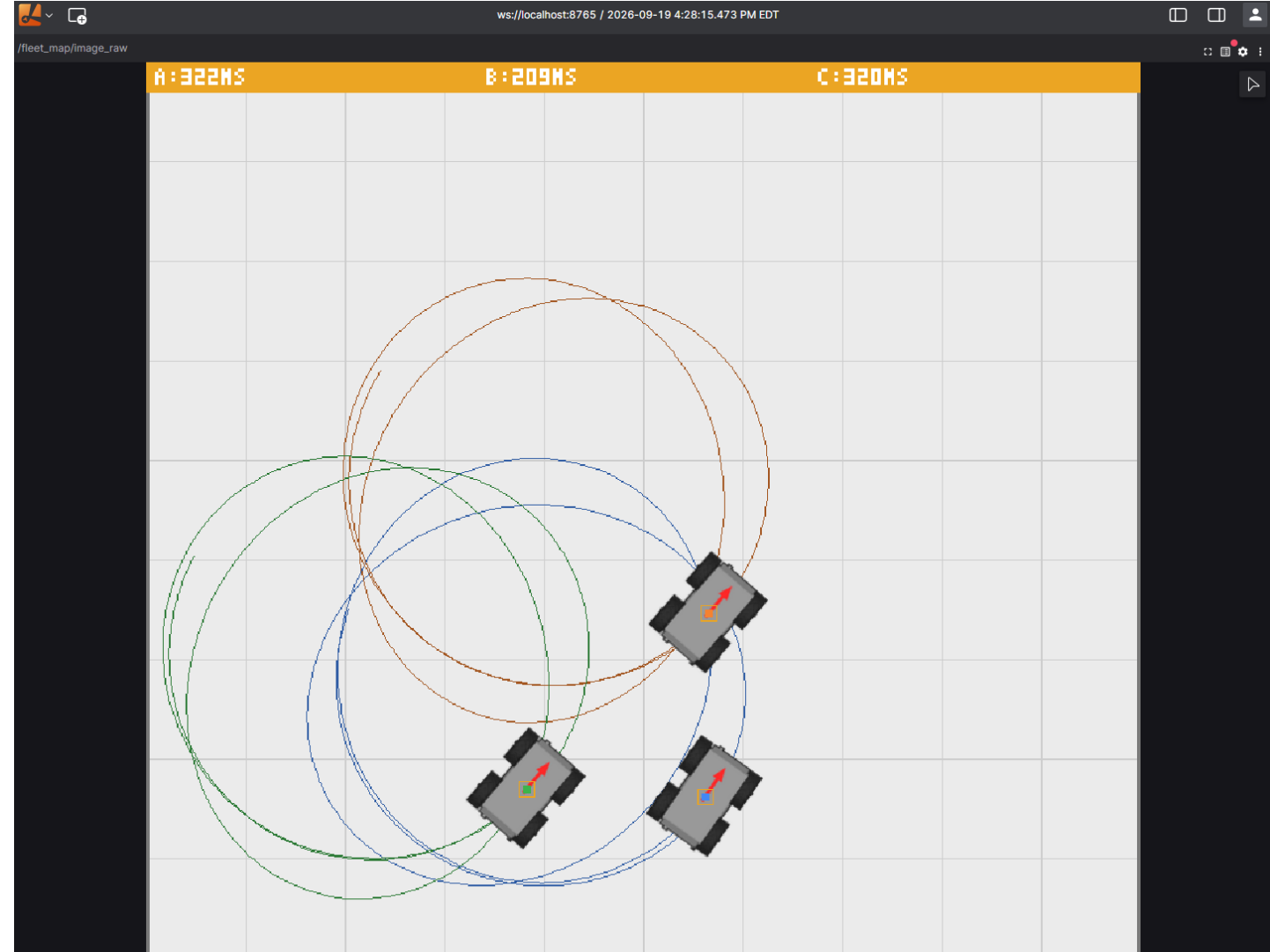


[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2\\_watch\\_a\\_healthy\\_link\\_fail.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2_watch_a_healthy_link_fail.md)

# RA | Exercise 2: Watch a Healthy Fleet Fail

- Discussion

- How did the robots perform under good vs bad network conditions?
- Is the fleet map sufficient to diagnose the issue?
- Did the data corroborate the observation?



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2\\_watch\\_a\\_healthy\\_link\\_fail.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/2_watch_a_healthy_link_fail.md)



# | Exercise 3: The Shared Medium

Target time: 15 minutes

- **Goal:** use the deployment from Exercise 2 to root cause the issue using a data-driven approach
  - The UI shows only part of the story
  - Is it a compute-availability issue or a network issue? Is it a configuration issue?
  - Impact of ROS 2 publisher QoS settings
  - Can you overload the network?

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/3\\_the\\_shared\\_medium.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/3_the_shared_medium.md)



# | Exercise 3: The Shared Medium

Target time: 15 minutes

- **Discussion: midway checkpoint**
  - What does the mock robot visualization tell you?
  - Did anyone run into host pressure?
  - Reliable vs best-effort configuration

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/3\\_the\\_shared\\_medium.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/3_the_shared_medium.md)

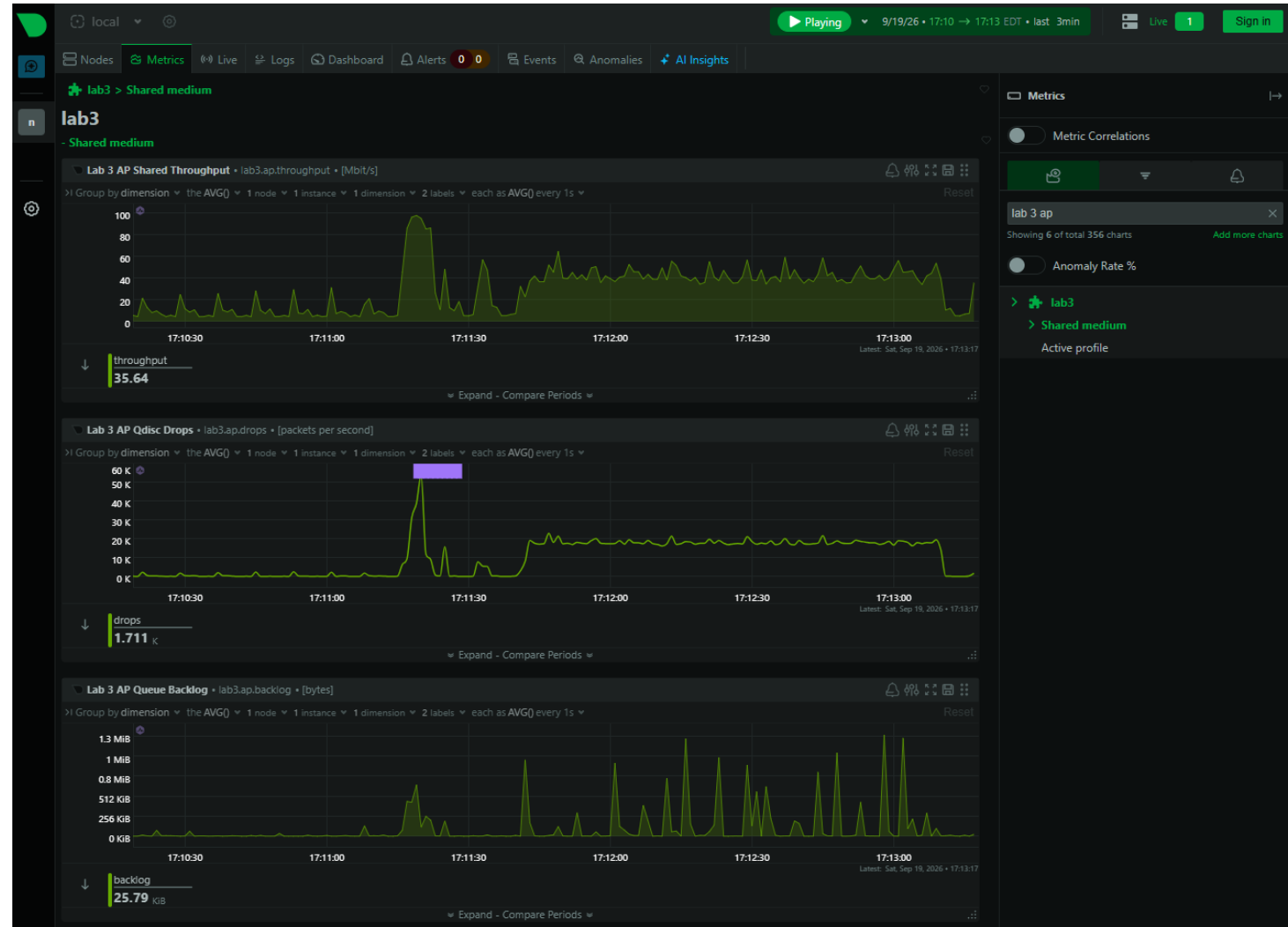


# | Exercise 3: The Shared Medium

Target time: 15 minutes

- Discussion

- Impact of subscribing to a heavy topic on a healthy/bad network link



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/3\\_the\\_shared\\_medium.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/3_the_shared_medium.md)



# | Transport Layer Impact

- The transport decides how a network problem shows up — as lost samples, or as late ones.
- Same ROS 2 graph, same data — UDP vs TCP behave very differently under stress.

## UDP — fire and forget

**No retransmission, no ordering guarantee:** a lost sample is simply gone.

Fits **high-rate, self-refreshing data** — sensor streams, telemetry, camera data.

Under loss you lose data, but the freshest sample still arrives on time.

## TCP — ordered and reliable

**Retransmits and re-orders**, so nothing is dropped — it's delayed instead.

**Head-of-line blocking:** one lost segment stalls everything behind it.

Fits **bulk, must-arrive traffic** — maps, configs, logs, file transfer, control.

## What do you expect to see?

Same fleet replay, same data — one runs over UDP, one over TCP.

Take a guess: which loses more samples, and which arrives late?

What happens to throughput and to p99 arrival gap under a degraded network?

We'll find out in the next exercise — and dig into it in Lab 4.

Reliability isn't free — you pay for it in latency



# | Exercise 4: Repeat Using TCP

Target time: 10 minutes

- **Goal:** explore the impact of using a reliable transport like TCP using the same deployment topology as Exercise 3
  - We are using Zenoh as the RMW in this exercise as it can be easily configured for TCP
  - Effect of QoS settings with TCP
  - Use CLI tools from Lab1 and Lab2 to validate symptoms with data

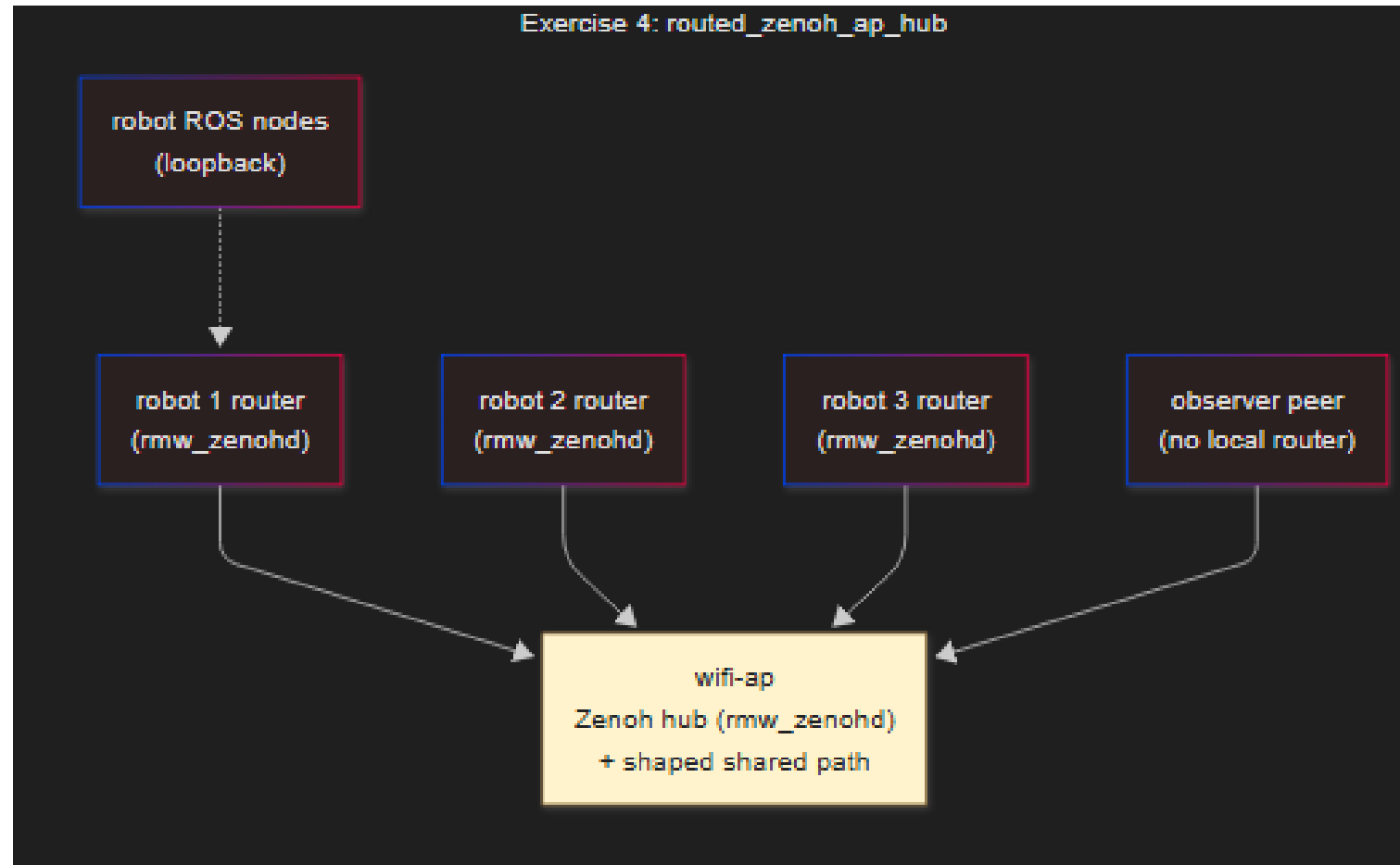
[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4\\_repeat\\_with\\_zenoh.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4_repeat_with_zenoh.md)



# Exercise 4: Repeat Using TCP

Target time: 10 minutes

- Discussion
  - Deployment topology



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4\\_repeat\\_with\\_zenoh.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4_repeat_with_zenoh.md)



# | Exercise 4: Repeat Using TCP

Target time: 10 minutes

- **Discussion: midway checkpoint**
  - What did you observe this time compared to Exercise 3?
  - Impact of changing QoS settings

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4\\_repeat\\_with\\_zenoh.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4_repeat_with_zenoh.md)

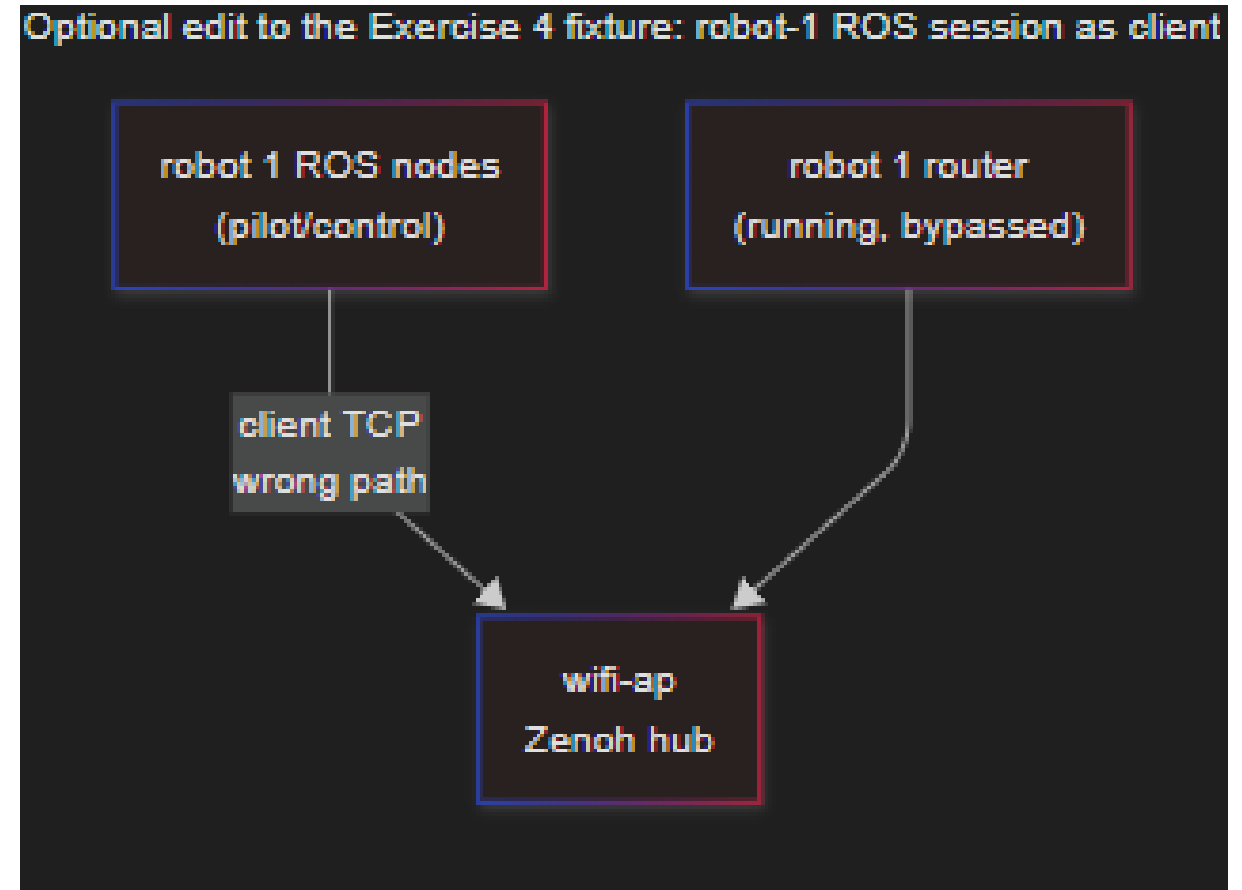


# Exercise 4: Repeat Using TCP

Target time: 10 minutes

- Discussion

- What happens when a local router isn't used?



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4\\_repeat\\_with\\_zenoh.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4_repeat_with_zenoh.md)



# | Exercise 5: After the Workshop

- **Goal:** explore technologies and techniques in further detail
  - Monitor your own robot or deployment
  - Create your own netem profiles
  - Use your own bags, to closer simulate your own expected deployments
- **Discussion**
  - Any wrap-up discussion on lab3 before moving on to lab4

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4\\_repeat\\_with\\_zenoh.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab3-stress-testing/exercises/4_repeat_with_zenoh.md)



# Lab 4: Benchmarking & Decision Matrix

# | Agenda

| Section              | Duration (minutes) |
|----------------------|--------------------|
| Intro                | 2                  |
| Pugh Decision Matrix | 5                  |
| Exercise 1           | 10                 |
| Exercise 2           | 5                  |
| Exercise 3           | 15                 |
| Wrap-up              | 3                  |



# | Decision Time

- **We know how to** configure RMWs, change deployment topologies, change transport layers, simulate real traffic
  - We have the knobs and the data
- **How do we decide** what to use for our use and deployment?

We need a method, not an opinion



# | Pugh Decision Matrix

- A structured way to compare options against **weighted criteria** — instead of arguing by intuition or headline numbers.
- Turns a multi-dimensional trade-off into one transparent, defensible score you can revisit as data changes.

## What it is

A **decision matrix**: candidate options in columns, evaluation criteria in rows.

Each **criterion carries a weight** (importance); each option gets a score per criterion.

Weighted scores are summed into a single total per option.

## Why we use it

Makes the criteria — and their relative importance — explicit and agreed up front.

Separates **measured evidence** from judgement calls, so bias is visible.

Gives a **repeatable, auditable rationale** we can defend and re-run later.

## How we use it

1. Agree criteria that matter for our workload.
2. Weight each criterion 1-5.
3. Score each option from benchmark data (or judgement, flagged †).
4. Sum weighted scores and rank — then sanity-check the winner.



# | Pugh Decision Matrix

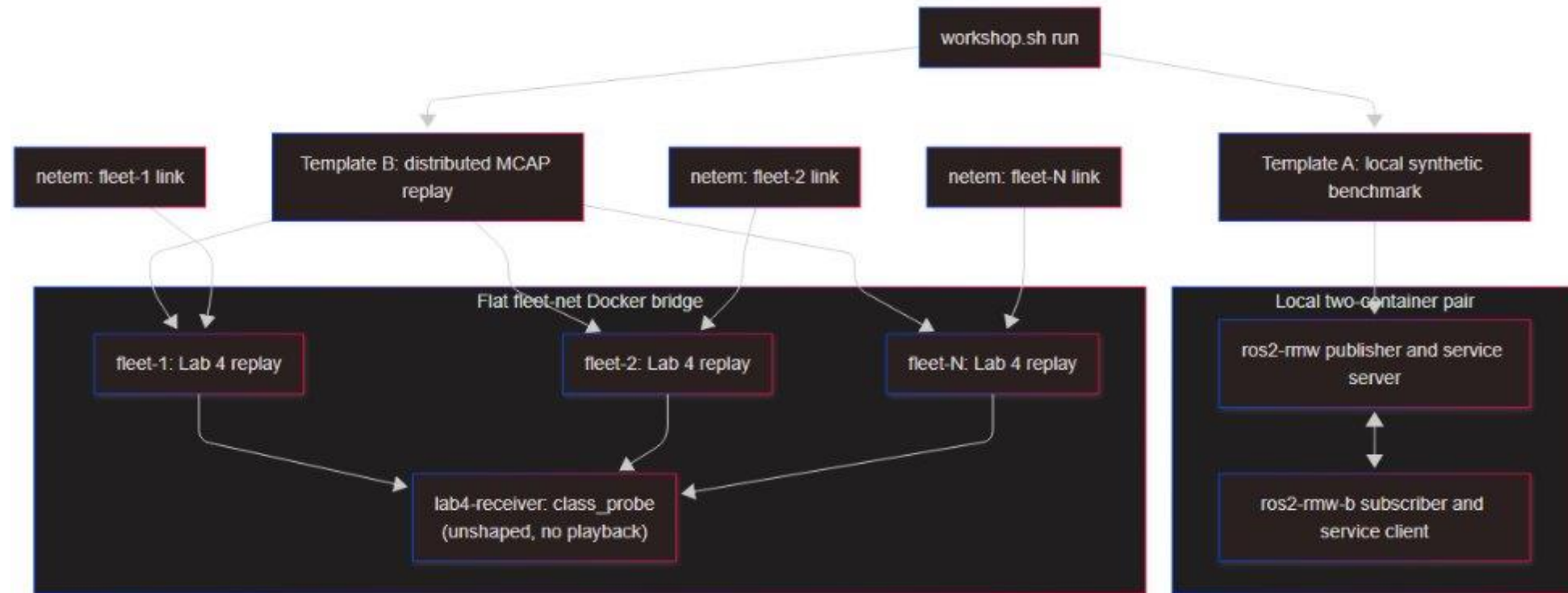
| Criterion                                      | W | Cyclone DDS  | Fast DDS    | Zenoh       |
|------------------------------------------------|---|--------------|-------------|-------------|
| Latency(p99) – Control arrival-gap p99 (ms)    | 5 | 5.0          | 1.4         | 1.0         |
| Data Freshness – State maximum silence (ms)    | 5 | 5.0          | 1.0         | 4.8         |
| Reliability – Estimated delivery shortfall (%) | 5 | 5.0          | 1.0         | 1.0         |
| Behaviour under loss/jitter (manual)           | 5 | 3.0†         | 3.0†        | 3.0†        |
| Throughput / bandwidth efficiency (Mb/s)       | 4 | 3.0          | 3.0         | 3.0         |
| CPU overhead – Receiver workload CPU (%)       | 3 | 5.0          | 1.0         | 3.7         |
| Ease of config & debugging (manual)            | 3 | 3.0†         | 3.0†        | 3.0†        |
| Discovery / connectivity robustness (manual)   | 4 | 3.0†         | 3.0†        | 3.0†        |
| <b>Weighted total</b>                          |   | <b>138.0</b> | <b>67.8</b> | <b>93.3</b> |
| <b>Ranking</b>                                 |   | <b>1st</b>   | <b>3rd</b>  | <b>2nd</b>  |



# Exercise 1: Benchmark RMWs

Target time: 10 minutes

- **Goal:** establish clean baselines
  - Use a controlled load and deployment, without applying network impairment



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/1\\_same\\_traffic\\_different\\_rmw.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/1_same_traffic_different_rmw.md)

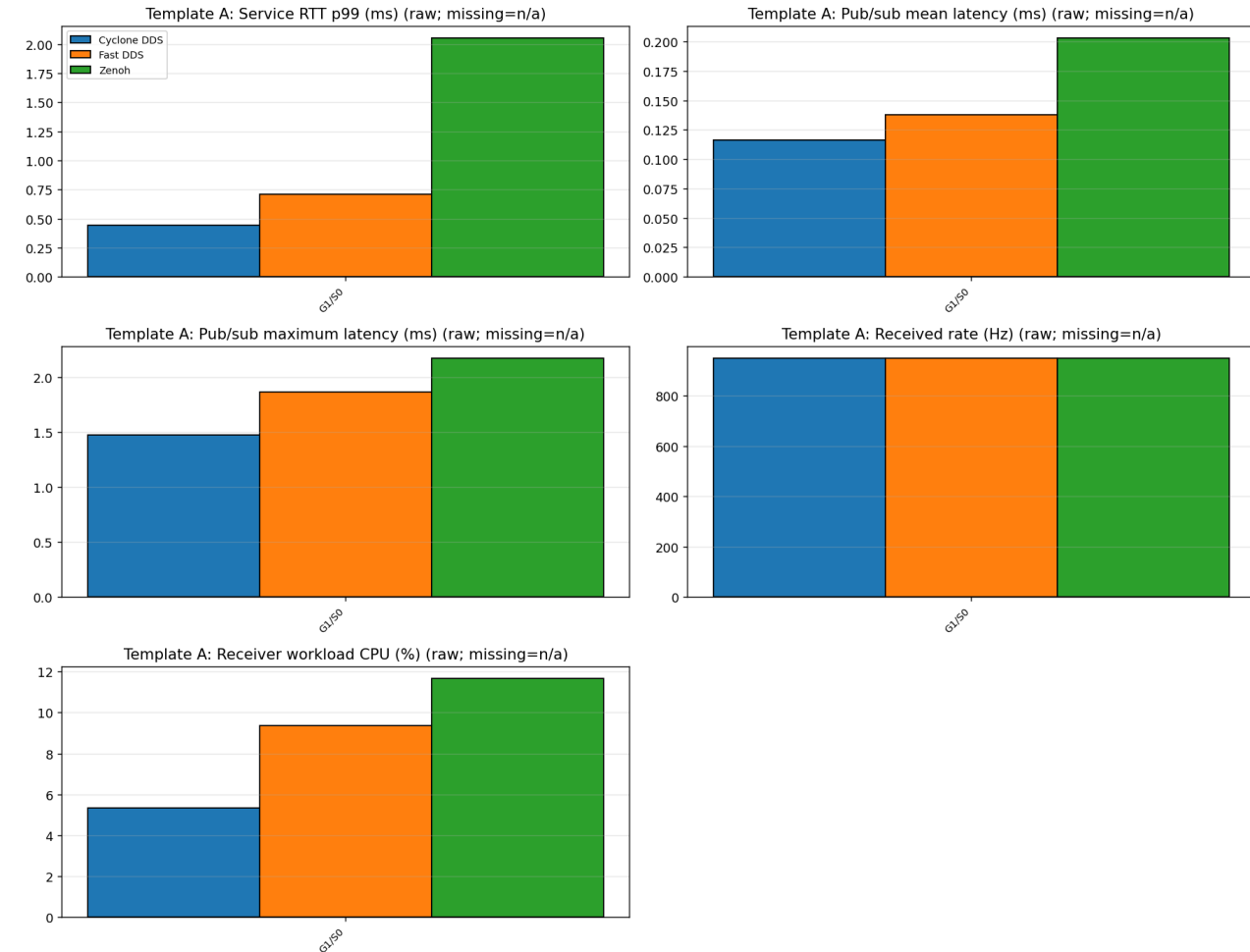


# Exercise 1: Benchmark RMWs

Target time: 10 minutes

- Discussion: template A
  - What do the graphs tell you?
  - Any unexpected results?

RMW comparison — raw observed values; no significance claim



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/1\\_same\\_traffic\\_different\\_rmw.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/1_same_traffic_different_rmw.md)

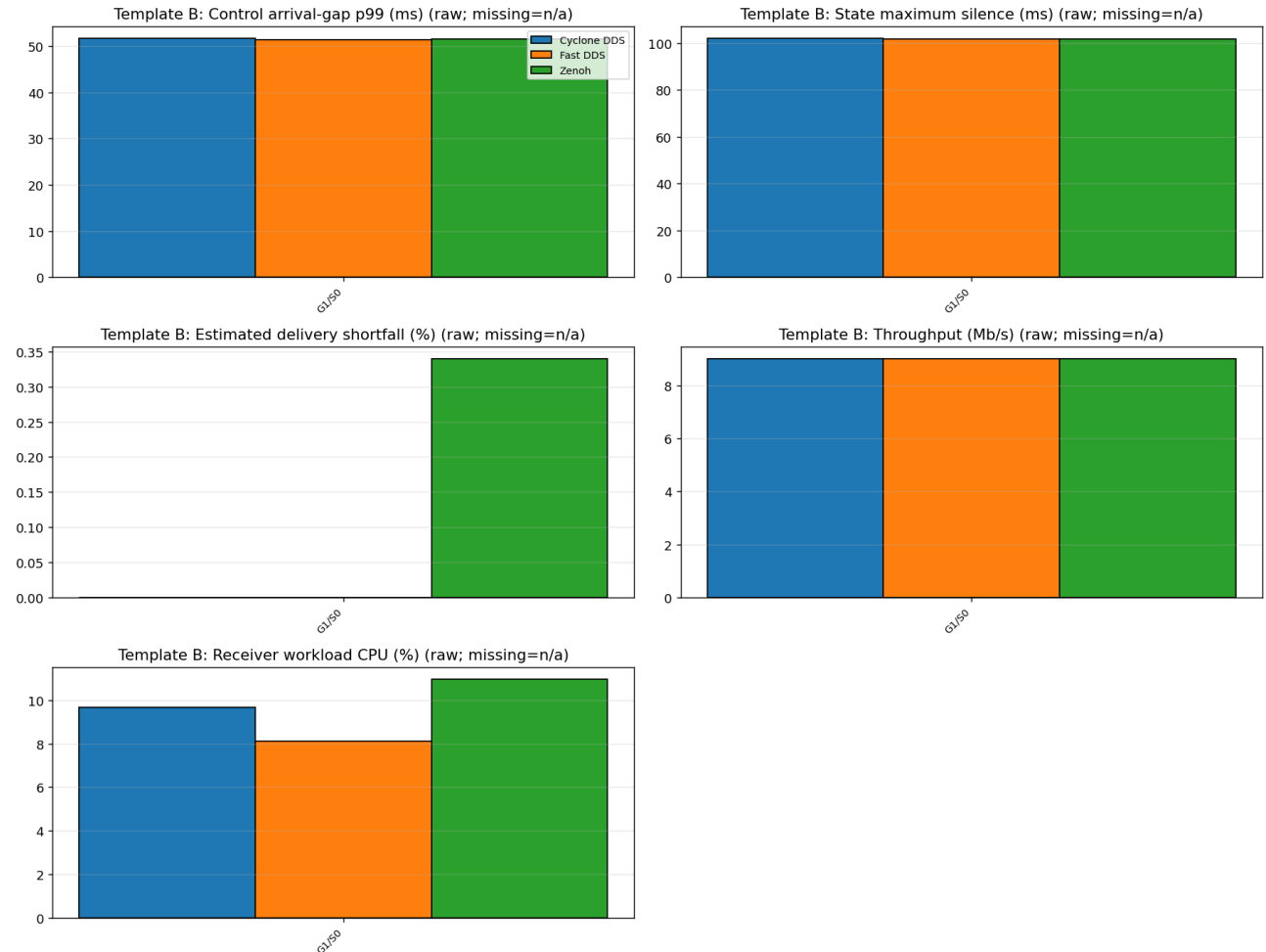


# Exercise 1: Benchmark RMWs

Target time: 10 minutes

- Discussion: template B
  - What do the graphs tell you?
  - Any unexpected results?

RMW comparison — raw observed values; no significance claim



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/1\\_same\\_traffic\\_different\\_rmw.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/1_same_traffic_different_rmw.md)



# | Exercise 2: Read the Matrix

Target time: 5 minutes

- **Goal:** define a Pugh matrix and use benchmark data to fill it in
  - Pugh matrix is pre-defined with criteria and weights; use as-is for this exercise, use as homework to create your own criteria and define your own weights
  - Understand how the data from Exercise 1 fits into the generated Pugh matrix

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2\\_read\\_the\\_matrix.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2_read_the_matrix.md)



# | Exercise 2: Read the Matrix

Target time: 5 minutes

- Discussion: Template A

- Evaluation criteria
  - Anything you'd add/change?
- Thoughts on the manual/objective criteria?
- How did you read the matrix? Do you agree with the results?

## Weighted Pugh decision matrix

### Template A — Two-Container Synthetic Baseline — Scenario S0

| Criterion                                              | W | Cyclone DDS      | Fast DDS         | Zenoh            |
|--------------------------------------------------------|---|------------------|------------------|------------------|
| Latency (p99) — Service RTT p99 (ms)                   | 4 | 5.0              | 4.3              | 1.0              |
| Data Freshness — Pub/sub mean latency (ms)             | 4 | 5.0              | 4.0              | 1.0              |
| Reliability (delivery) — Pub/sub maximum latency (ms)  | 3 | 5.0              | 2.8              | 1.0              |
| Behaviour under loss/jitter ( <i>manual</i> )          | 1 | 3.0 <sup>+</sup> | 3.0 <sup>+</sup> | 3.0 <sup>+</sup> |
| Throughput / bandwidth efficiency — Received rate (Hz) | 4 | 3.0              | 3.0              | 3.0              |
| CPU overhead — Receiver workload CPU (%)               | 5 | 5.0              | 2.5              | 1.0              |
| Ease of config & debugging ( <i>manual</i> )           | 4 | 3.0 <sup>+</sup> | 3.0 <sup>+</sup> | 3.0 <sup>+</sup> |
| Discovery / connectivity robustness ( <i>manual</i> )  | 2 | 3.0 <sup>+</sup> | 3.0 <sup>+</sup> | 3.0 <sup>+</sup> |
| <b>Weighted total</b>                                  |   | <b>113.0</b>     | <b>86.9</b>      | <b>49.0</b>      |

**Ranking:** Cyclone DDS (113.0), Fast DDS (86.9), Zenoh (49.0)

<sup>+</sup> provisional neutral manual score; <sup>‡</sup> inherited legacy template-wide manual score.

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2\\_read\\_the\\_matrix.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2_read_the_matrix.md)



# | Exercise 2: Read the Matrix

Target time: 5 minutes

- Discussion: Template B
  - Compare to Template A results

## Weighted Pugh decision matrix

### Template B — Fleet MCAP Replay — Scenario S0

| Criterion                                                 | W | Cyclone DDS | Fast DDS     | Zenoh       |
|-----------------------------------------------------------|---|-------------|--------------|-------------|
| Latency (p99) — Control arrival-gap p99 (ms)              | 5 | 1.0         | 5.0          | 3.3         |
| Data Freshness — State maximum silence (ms)               | 5 | 1.0         | 3.8          | 5.0         |
| Reliability (delivery) — Estimated delivery shortfall (%) | 5 | 5.0         | 5.0          | 1.0         |
| Behaviour under loss/jitter ( <i>manual</i> )             | 5 | 3.0+        | 3.0+         | 3.0+        |
| Throughput / bandwidth efficiency — Throughput (Mb/s)     | 4 | 3.0         | 3.0          | 3.0         |
| CPU overhead — Receiver workload CPU (%)                  | 3 | 2.8         | 5.0          | 1.0         |
| Ease of config & debugging ( <i>manual</i> )              | 3 | 3.0+        | 3.0+         | 3.0+        |
| Discovery / connectivity robustness ( <i>manual</i> )     | 4 | 3.0+        | 3.0+         | 3.0+        |
| <b>Weighted total</b>                                     |   | <b>91.5</b> | <b>132.2</b> | <b>97.6</b> |

**Ranking:** Fast DDS (132.2), Zenoh (97.6), Cyclone DDS (91.5)

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2\\_read\\_the\\_matrix.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2_read_the_matrix.md)



# | Exercise 2: Read the Matrix

Target time: 5 minutes

- Discussion

- Any situation where you'd find this decision matrix useful
- Any other questions?

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2\\_read\\_the\\_matrix.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/2_read_the_matrix.md)



# | Exercise 3: Stressed vs Clean

Target time: 15 minutes

- **Goal:** stress the network and re-run the benchmarks
  - Understand how a RMW, and its configuration, react under different network conditions
  - Compare clean and stressed results side-by-side

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/3\\_stressed\\_vs\\_clean.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/3_stressed_vs_clean.md)



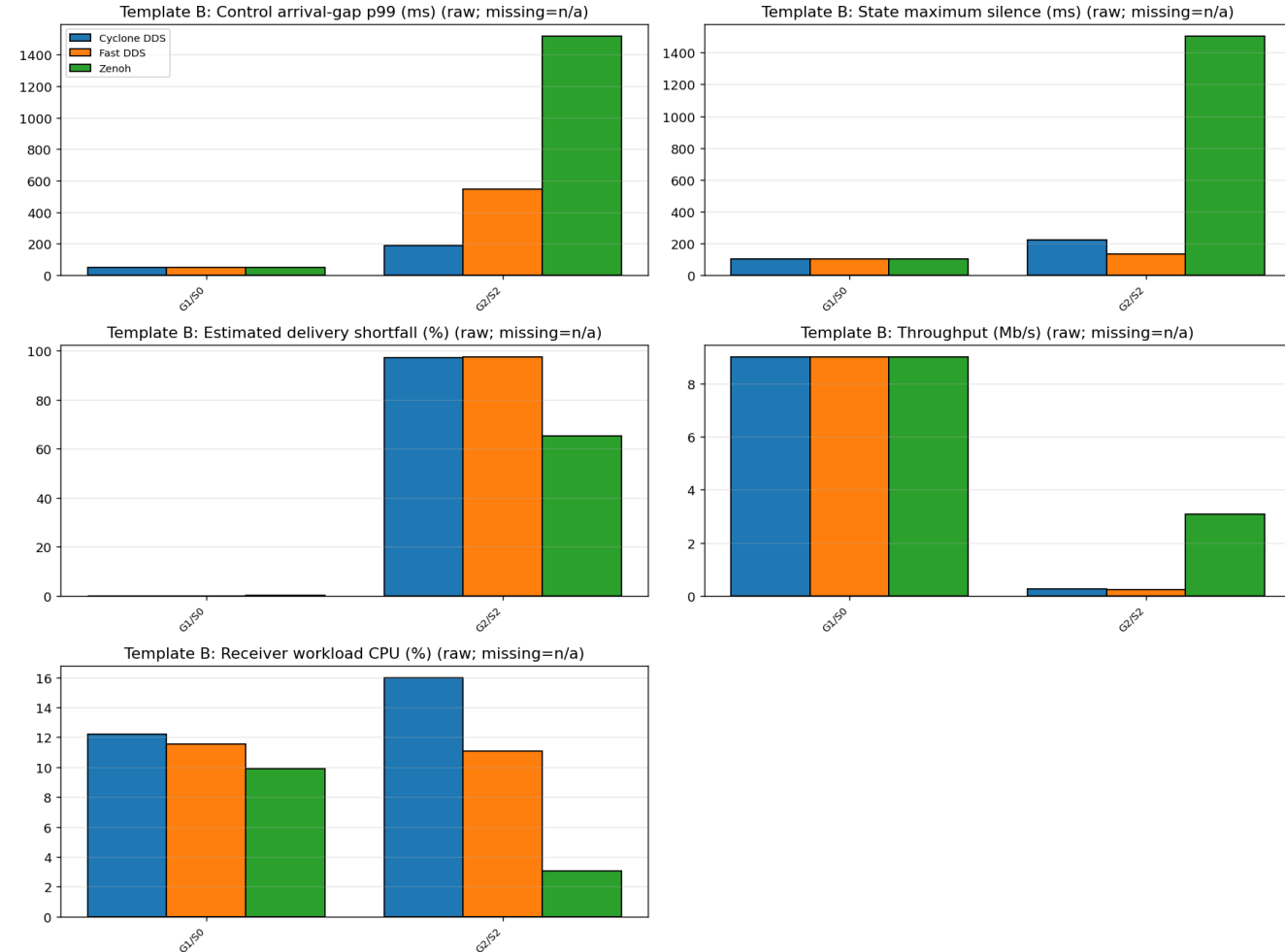
# Exercise 3: Stressed vs Clean

Target time: 15 minutes

- Discussion: benchmark results

- How does stressing the network impact the results?

RMW comparison — raw observed values; no significance claim



[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/3\\_stressed\\_vs\\_clean.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/3_stressed_vs_clean.md)



# | Exercise 3: Stressed vs Clean

Target time: 15 minutes

- Discussion: Pugh decision matrix
  - Changes compared to a clean network?

## Template B — Fleet MCAP Replay — Scenario S2

| Criterion                                                          | W | Cyclone DDS | Fast DDS    | Zenoh        |
|--------------------------------------------------------------------|---|-------------|-------------|--------------|
| Latency (p99) — Control arrival-gap p99 (ms)                       | 5 | 5.0         | 3.9         | 1.0          |
| Data Freshness — State maximum silence (ms)                        | 5 | 4.7         | 5.0         | 1.0          |
| Reliability (delivery) — Estimated delivery shortfall (%)          | 5 | 1.0         | 1.0         | 5.0          |
| Behaviour under loss/jitter ( <i>manual</i> )                      | 5 | 3.0+        | 3.0+        | 3.0+         |
| Throughput / bandwidth efficiency — Throughput (Mb/s)              | 4 | 1.0         | 1.0         | 5.0          |
| CPU overhead — Receiver workload CPU (%)                           | 3 | 1.0         | 2.5         | 5.0          |
| Ease of config & debugging ( <i>manual</i> )                       | 3 | 3.0+        | 3.0+        | 3.0+         |
| Discovery / connectivity robustness ( <i>manual</i> )              | 4 | 3.0+        | 3.0+        | 3.0+         |
| <b>Weighted total</b>                                              |   | <b>97.0</b> | <b>97.2</b> | <b>106.0</b> |
| <b>Ranking:</b> Zenoh (106.0), Fast DDS (97.2), Cyclone DDS (97.0) |   |             |             |              |

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/3\\_stressed\\_vs\\_clean.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/3_stressed_vs_clean.md)

# | Exercises 4 & 5: After the Workshop

- **Goal:** apply the benchmarking and Pugh matrix in your own decision workflow
  - Bring your own deployment's more realistic packet capture or simulation
  - Use different deployment topologies and RMW configuration (e.g. a supplied low-latency-tuned Zenoh variant is available in the workshop)
  - Change the weights and re-generate the Pugh matrices
- **Discussion**
  - Any wrap-up discussion on lab4

[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/4\\_generate\\_your\\_own\\_data.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/4_generate_your_own_data.md)  
[https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/5\\_after\\_the\\_workshop.md](https://github.com/clearpathrobotics/roscon2026-workshop-mastering-the-jazzy-rmw/blob/main/lab4-benchmarking-and-decision/exercises/5_after_the_workshop.md)



# Summary

# | Configure → Measure → Stress → Decide

## Configure

- one RMW, many configs

## Measure

- the wire shows what really happens

## Stress

- AP drops and backlog reveal the weak link

## Decide

- weights turn data into a choice

 | No single RMW wins for every system.

# | How do people apply this to their projects?

The recipe in these exercises does not depend on the mock fleet:  
bring up your own robot under your chosen RMW config,  
record a bag of whatever it actually does,  
and run it through the same instruments.

Lab 4 – Record an MCAP, run it through Template B exactly as Lab 4 Exercise 4

Lab 2 – Record a PCAP, Analyze in Webshark

# | Monday: prove it on your own system.

The deeper project. Lab 4 exercises 4 and 5 walk you through it.

## Capture

- your own MCAP

## Replay

- under your network profile

## Score

- the manual criteria for real

## Re-run

- when topology or RMW version changes



# | Try Webshark on your own robot's traffic this week.

Capture on the robot, open the pcap in Webshark on your laptop. Works today.

It's straightforward:

- capture with tshark on the robot
- open it in the Lab 2 viewer
- no new tooling to learn

What you'll see:

- discovery traffic
- QoS advertised in SEDP
- who's chatty, what retransmits

# | Go Deeper

- This repo — everything linked inside (star it)
- SROS 2 — encrypt the data you just watched in Webshark · <https://github.com/ros2/sros2>
- performance\_test (Apex.AI) — [https://gitlab.com/ApexAI/performance\\_test](https://gitlab.com/ApexAI/performance_test)
- ros2-performance (iRobot) — <https://github.com/irobot-ros/ros2-performance>
-  Communications & Performance — Thu 1:50pm, ROSCon 2026



# | Q & A and Survey Link





# Thank you

[www.rockwellautomation.com](http://www.rockwellautomation.com)

